CloudKit as Your Backend

From iOS to Server-Side Swift

Leo Dion @leogdion@c.im

iPhone iPhold

16GB
$2999

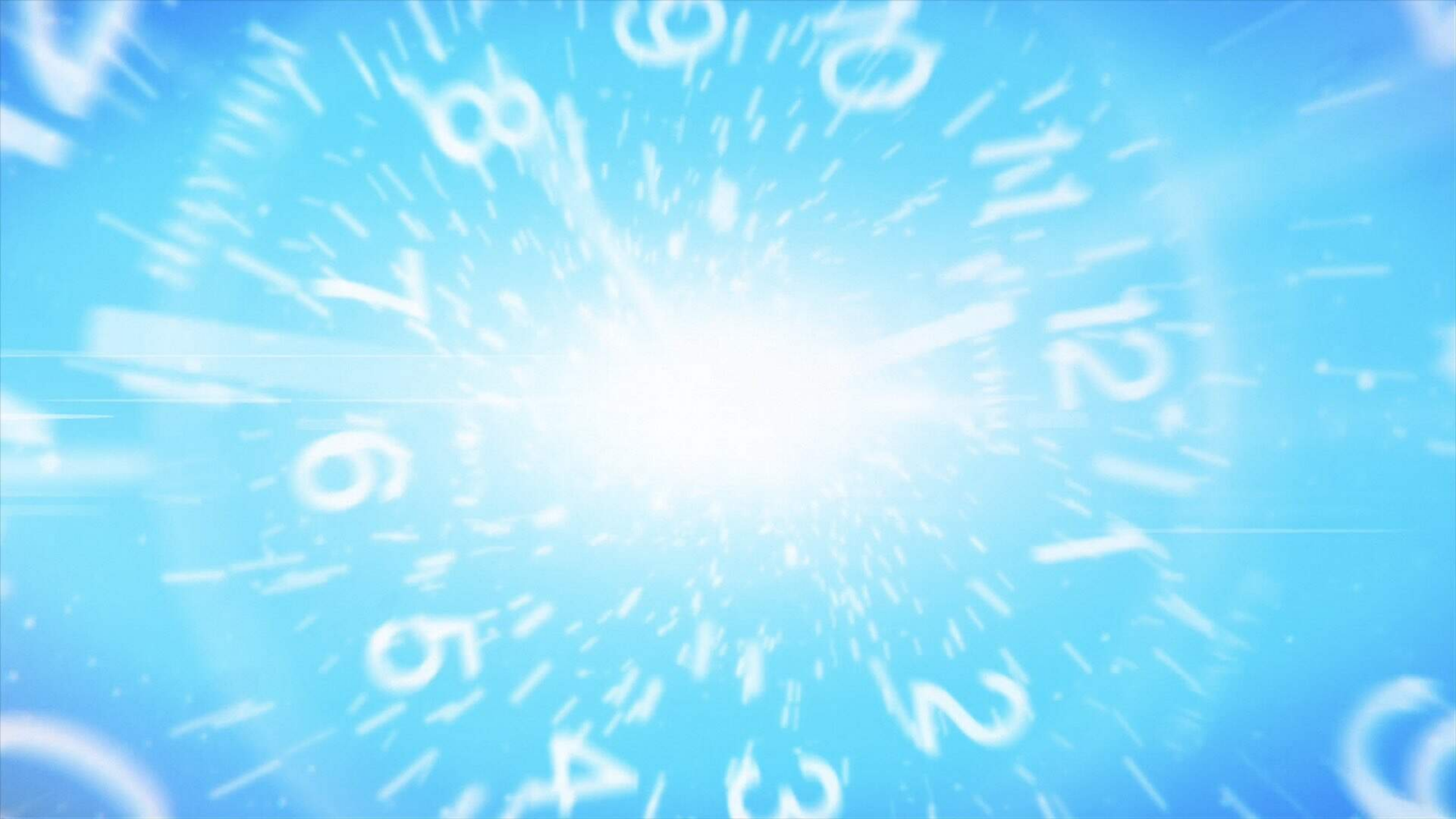
CloudKit as Your Backend

From iOS to Server-Side Swift

Leo Dion @leogdion@c.im

What is CloudKit

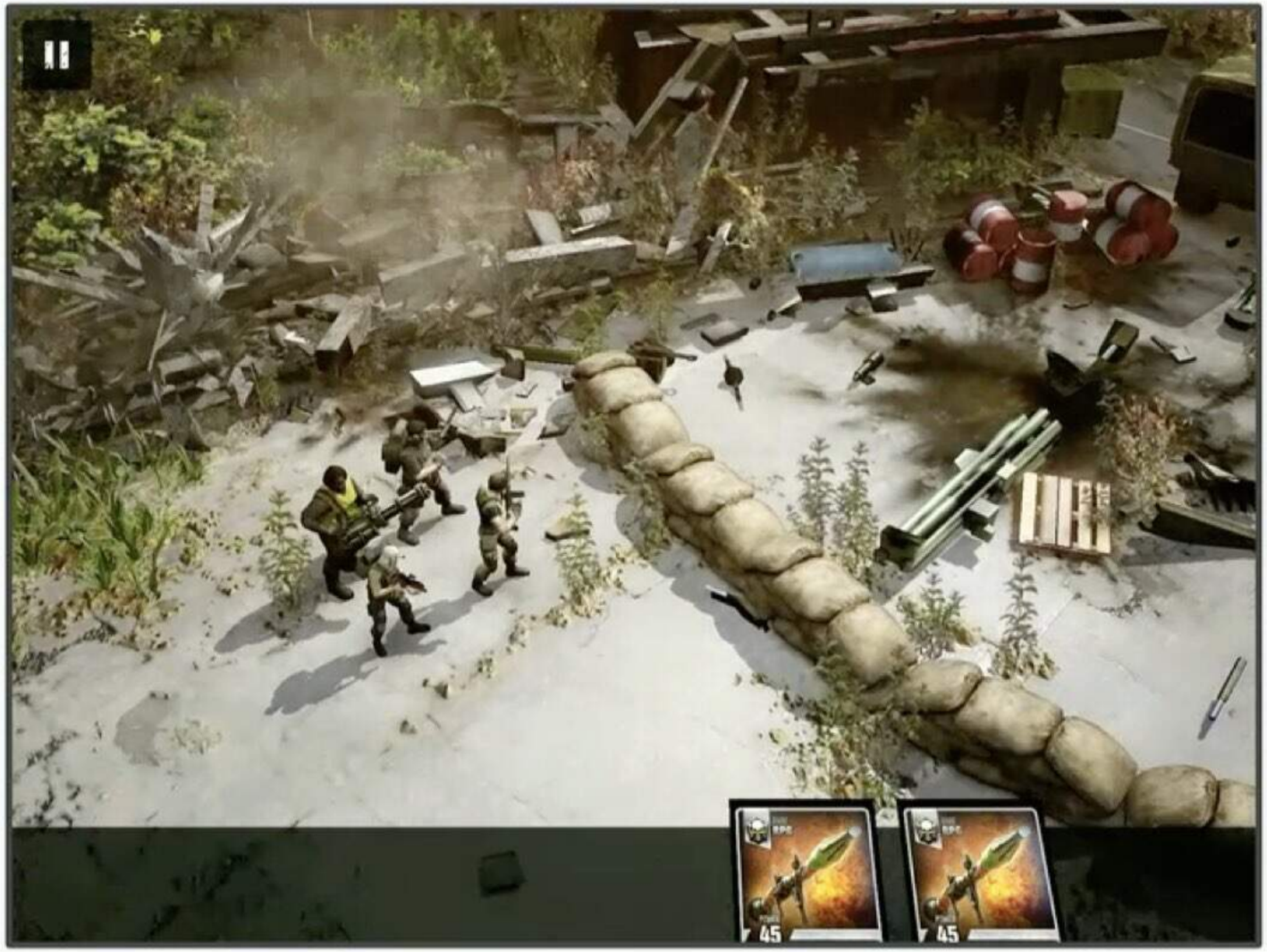
@leogdion@c.im




WWDC
2014

WWDC

New programming language


45
45

CloudKit




@leogdion@c.im

@leogdion@c.im

@leogdion@c.im

Bundle Identifier com.brightdigit.MistDemoApp

Provisioning Profile Xcode Managed Profile

Signing Certificate Apple Development: C I (78LK5545WT)

iCloud

Services
- [ ] Key-value storage
- [ ] iCloud Documents
- [x] CloudKit

Containers
- [x] iCloud.com.brightdigit.MistDemo
- [ ] iCloud.com.brightdigit.Bushel
- [ ] iCloud.com.brightdigit.Celestra

CloudKit Console

Add capabilities by clicking the "+" button above.

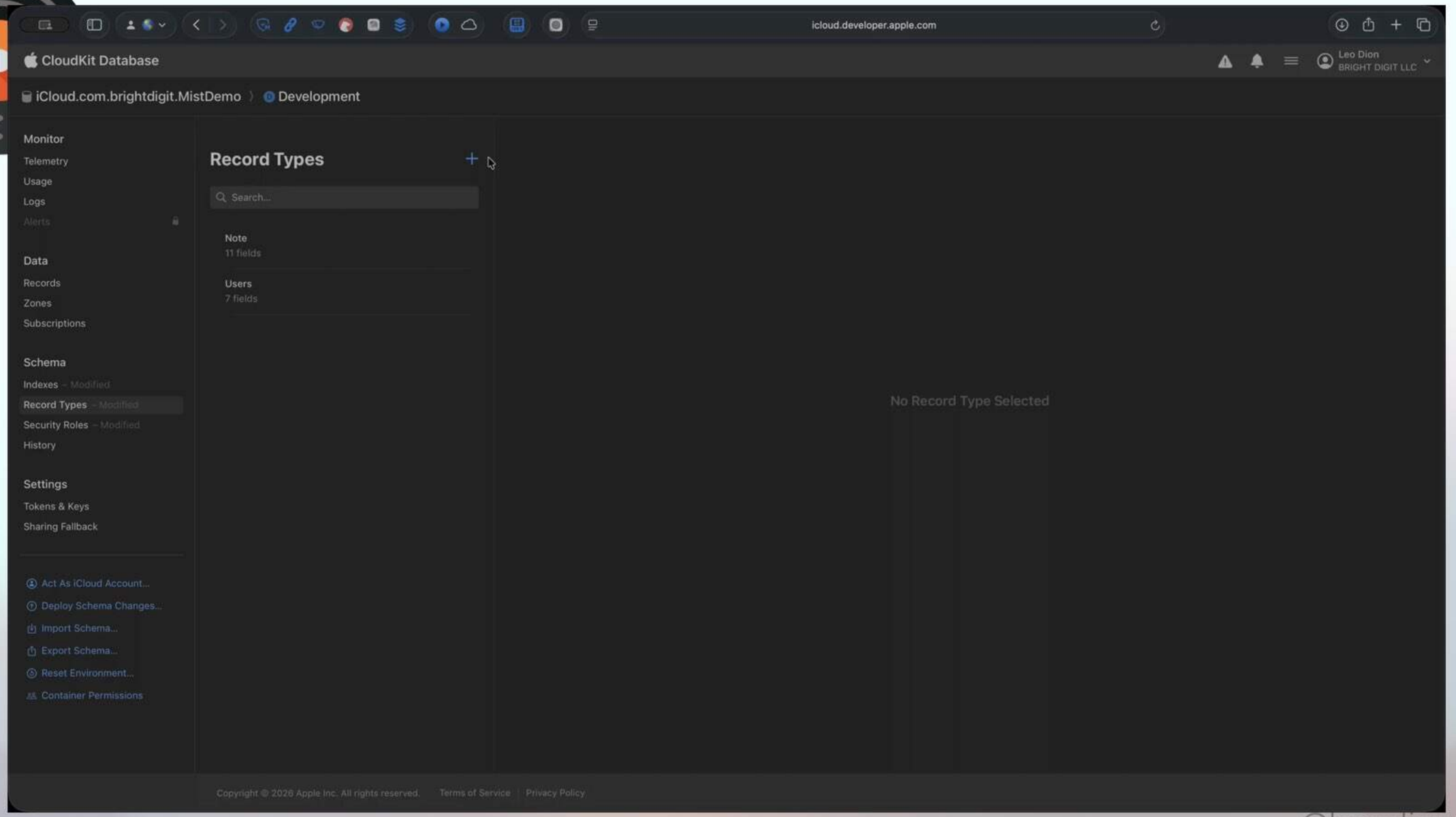
@leogdion@c.im

@leogdion@c.im

@leogdion@c.im

CloudKit Console

@leogdion@c.im



@leogdion@c.im

String

Int

NSNumber

Double

Bytes

Data

Date

Location

CLLocation

Reference

CKReference

Asset

CKAsset

List

Array<>

@leogdion@c.im

```
RECORD TYPE Note (
    "title"                 STRING QUERYABLE SORTABLE SEARCHABLE,
    "index"                 INT64 QUERYABLE SORTABLE,
    "image"                 ASSET
);
```

https://developer.apple.com/documentation/cloudkit/integrating-a-text-based-schema-into-your-workflow

@leogdion@c.im

icloud.developer.apple.com

Leo Dion
BRIGHT DIGIT LLC

igit.MistDemo 〉 Development

Logs
Alerts

Data
Records
Zones
Subscriptions

Schema
Indexes – Modified
Record Types – Modified
Security Roles – Modified
History

Settings
Tokens & Keys
Sharing Fallback

Act As iCloud Account...
Deploy Schema Changes...
Import Schema...
Export Schema...
Reset Environment...
Container Permissions

Records

Private Database | _defaultZone | Query Records | Saved Queries

RECORD TYPE Note | FIELDS All | Add filter or sort to query | Query Records

Name	Type	createdAt	image	index	modified	title	Ch. Tag	Created	Modified
08a6de7f-b33a-4661-88	Note		Binary file (3546.10KB)			Uploaded Image - 2/3/...	ml6qunmp	2/3/2026, 3:18:56 PM ...	2/3/2026, 3:18:56 PM ...
18c865fa-cf98-40f8-9c	Note		Binary file (3546.10KB)			Uploaded Image - 2/3/...	ml6pcpzf	2/3/2026, 2:37:00 PM ...	2/3/2026, 2:37:00 PM ...
1df02d58-3d33-4de0-af	Note		Binary file (3546.10KB)			Uploaded Image - 2/3/...	ml6nhmbw	2/3/2026, 1:44:49 PM ...	2/3/2026, 1:44:49 PM ...
dfa4e155-277c-4af6-bcf	Note		Binary file (3546.10KB)			Uploaded Image - 2/3/...	ml6p7b51	2/3/2026, 2:32:47 PM ...	2/3/2026, 2:32:47 PM ...
dfc308c4-fc1e-48a2-8c	Note		Binary file (3546.10KB)			Uploaded Image - 2/3/...	ml6oz9mm	2/3/2026, 2:26:32 PM ...	2/3/2026, 2:26:32 PM ...
f1da1b74-7819-4b41-825	Note		Binary file (3546.10KB)			Uploaded Image - 2/2/...	ml5err7p	2/2/2026, 4:52:59 PM ...	2/2/2026, 4:52:59 PM ...
note-1769721369-7517	Note					My Note	mkzyerys	1/29/2026, 9:16:09 PM...	1/29/2026, 9:16:09 PM...
note-1769721441-4772	Note					My New Note	mkzygbpg	1/29/2026, 9:17:21 PM ...	1/29/2026, 9:17:21 PM ...
note-1770050209-4402	Note		Binary file (3546.10KB)			My Note	ml5e6yn4	2/2/2026, 4:36:49 PM ...	2/2/2026, 4:52:40 PM ...
note-1770050321-8069	Note					My Note	ml5e9d0a	2/2/2026, 4:38:41 PM ...	2/2/2026, 4:38:41 PM ...
note-1770050552-6991	Note					My Note	ml5eebbp	2/2/2026, 4:42:32 PM ...	2/2/2026, 4:42:32 PM ...
note-1770125672-6945	Note					My Note	ml6n4ear	2/3/2026, 1:34:32 PM ...	2/3/2026, 1:34:32 PM ...

12 items

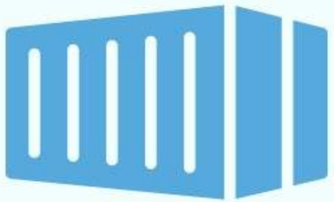



@leogdion@c.im

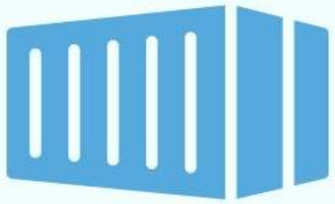
Databases

@leogdion@c.im

Databases



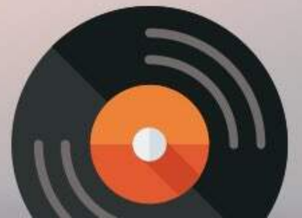
@leogdion@c.im

Container 

Environment 

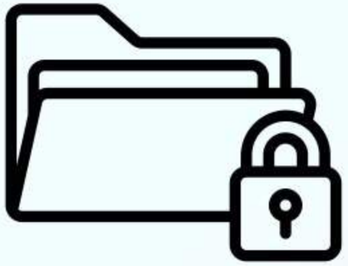
Databases 

Zones 

Records 

@leogdion@c.im

Private Database



Databases

Public Database

@leogdion@c.im

Private Database

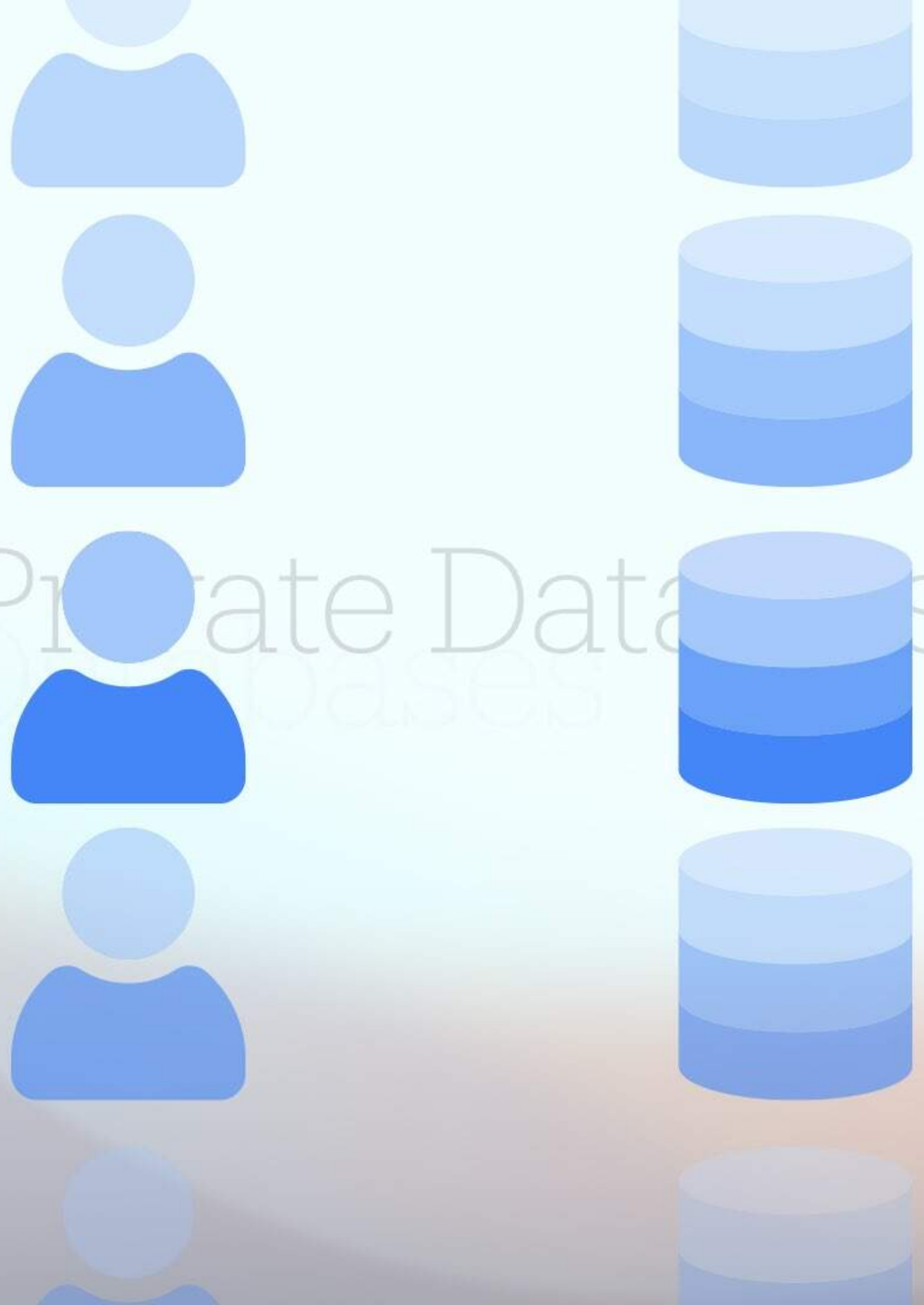
Public Database

@leogdion@c.im

Private Database

Public Database

@leogdion@c.im



Public Database

@leogdion@c.im

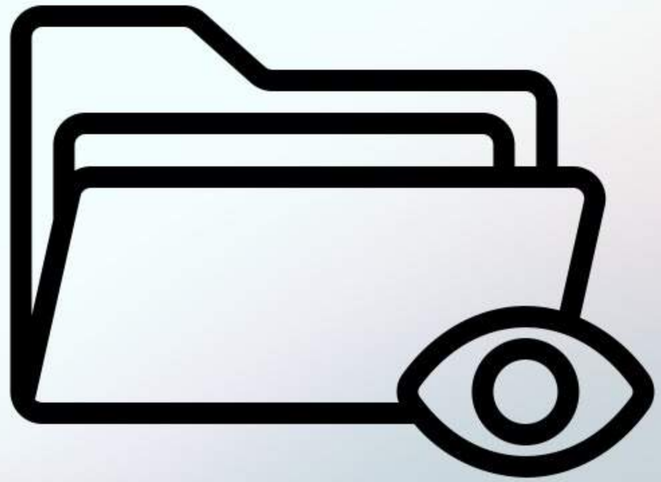
Private Database

Public Database

@leogdion@c.im

Private Database

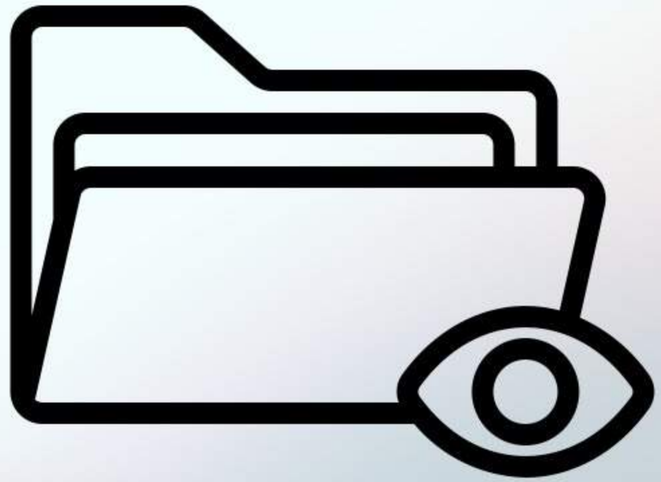
Public Database



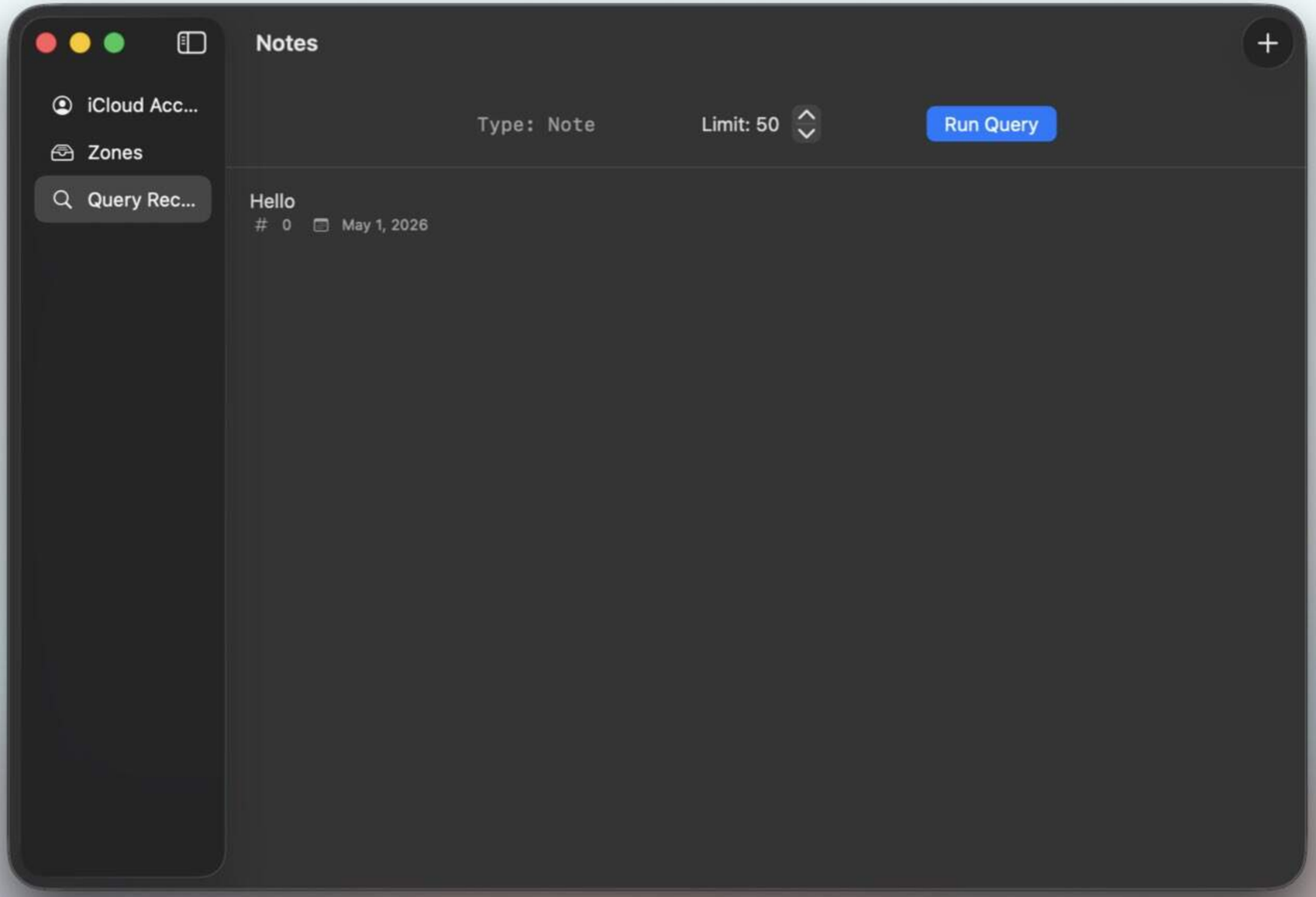
@leogdion@c.im

Private Database

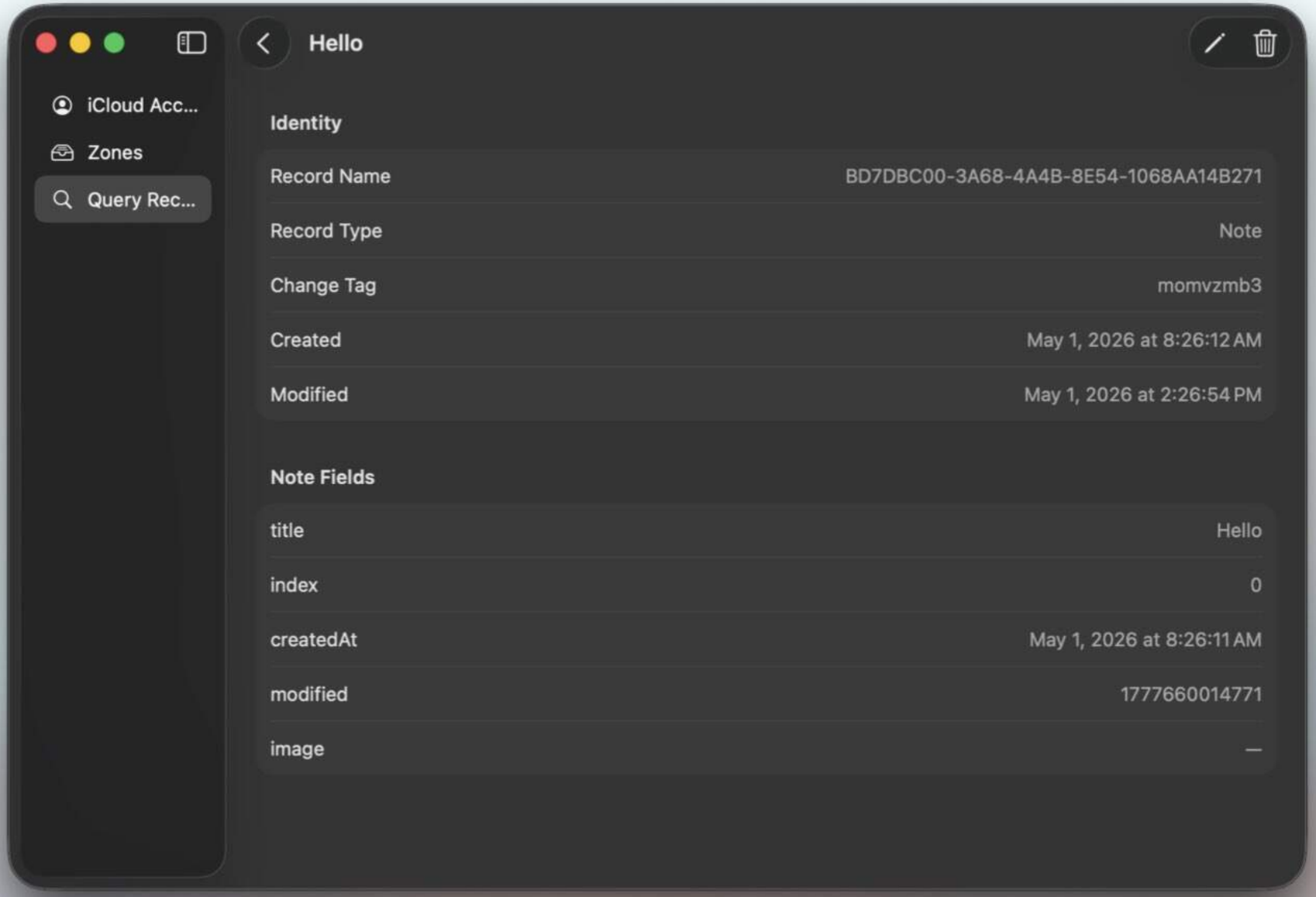
Public Database



@leogdion@c.im



@leogdion@c.im



@leogdion@c.im

What We'll Talk About

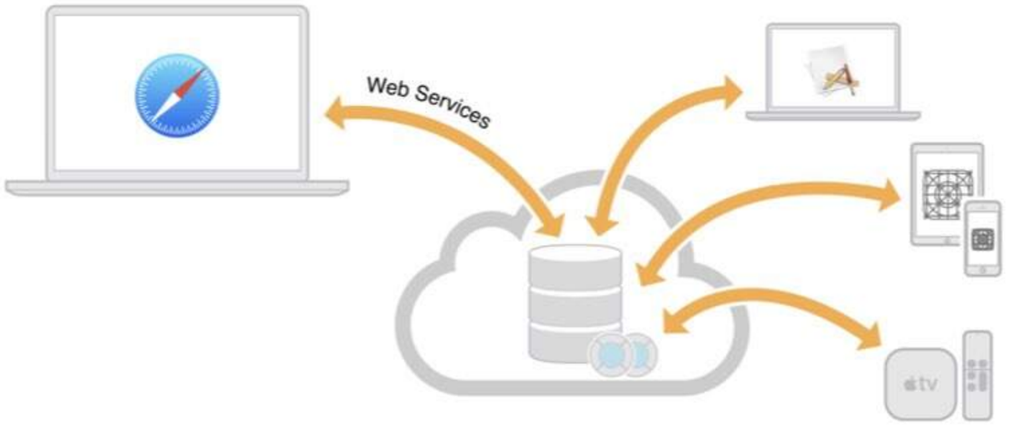
- What is CloudKit Web Services?
- Why Server Side CloudKit?
- How Server Side CloudKit?
  - Coding CloudKit Web Services
  - Converting Documentation to ~~Implementation~~ Documentation
  - Swift Challenges
- What's Next?

CloudKit Web Services

@leogdion@c.im

About CloudKit Web Services

You use the CloudKit native framework to take your app's existing data and store it in the cloud so that the user can access it on multiple devices. Then you can use CloudKit web services or CloudKit JS to provide a web interface for users to access the same data as your app. To use CloudKit web services, you must have the schema for your databases already created. CloudKit web services provides an HTTP interface to fetch, create, update, and delete records, zones, and subscriptions. You also have access to discoverable users and contacts. Alternatively, you can use the JavaScript API to access these services from a web app.



This document assumes that you are already familiar with CloudKit and CloudKit Dashboard. The following resources provide more information about CloudKit:

- *CloudKit Quick Start* and *CloudKit Framework Reference* teach you how to create a CloudKit app and use CloudKit Dashboard.
- *CloudKit JavaScript Reference* describes an alternative JavaScript API for accessing your app's CloudKit databases from a web app.
- *CloudKit Catalog: An Introduction to CloudKit (Cocoa and JavaScript)* sample code demonstrates CloudKit web services and CloudKit JS. For the interactive hosted version of this sample, go to CloudKit Catalog.

Framework

CloudKit JS

Provide access from your web app to your CloudKit app's containers and databases.

CloudKit JS 1.0+

Overview

Use CloudKit JS to build a web interface that lets users access the same public and private databases as your CloudKit app running on iOS or macOS. You must have an existing CloudKit app and enable web services to use CloudKit JS.

Composing Web Service Requests

CloudKit web service URLs have common components. Always begin the endpoints with the CloudKit web service path followed by `database`, the protocol version number, container ID, and environment. Then pass the operation subpath containing the credentials to access CloudKit using either an API Token or server-to-server key.

Use an API Token from a website or an embedded web view in a native app, or when you need to authenticate the user, described in Accessing CloudKit Using an API Token. If the request requires user authentication, use the information in the response to authenticate the user. Use a server-to-server key to access the public database from a server process or script, described in Accessing CloudKit Using a Server-to-Server Key.

Read the following chapters for the operation-specific JSON request and response dictionaries associated with each web service call.

The Web Service URL

This is the path and common parameters used in all the CloudKit web service URLs:

```
[path]/database/[version]/[container]/[environment]/[operation-specific subpath]
```

CloudKit Web Services

@leogdion@c.im

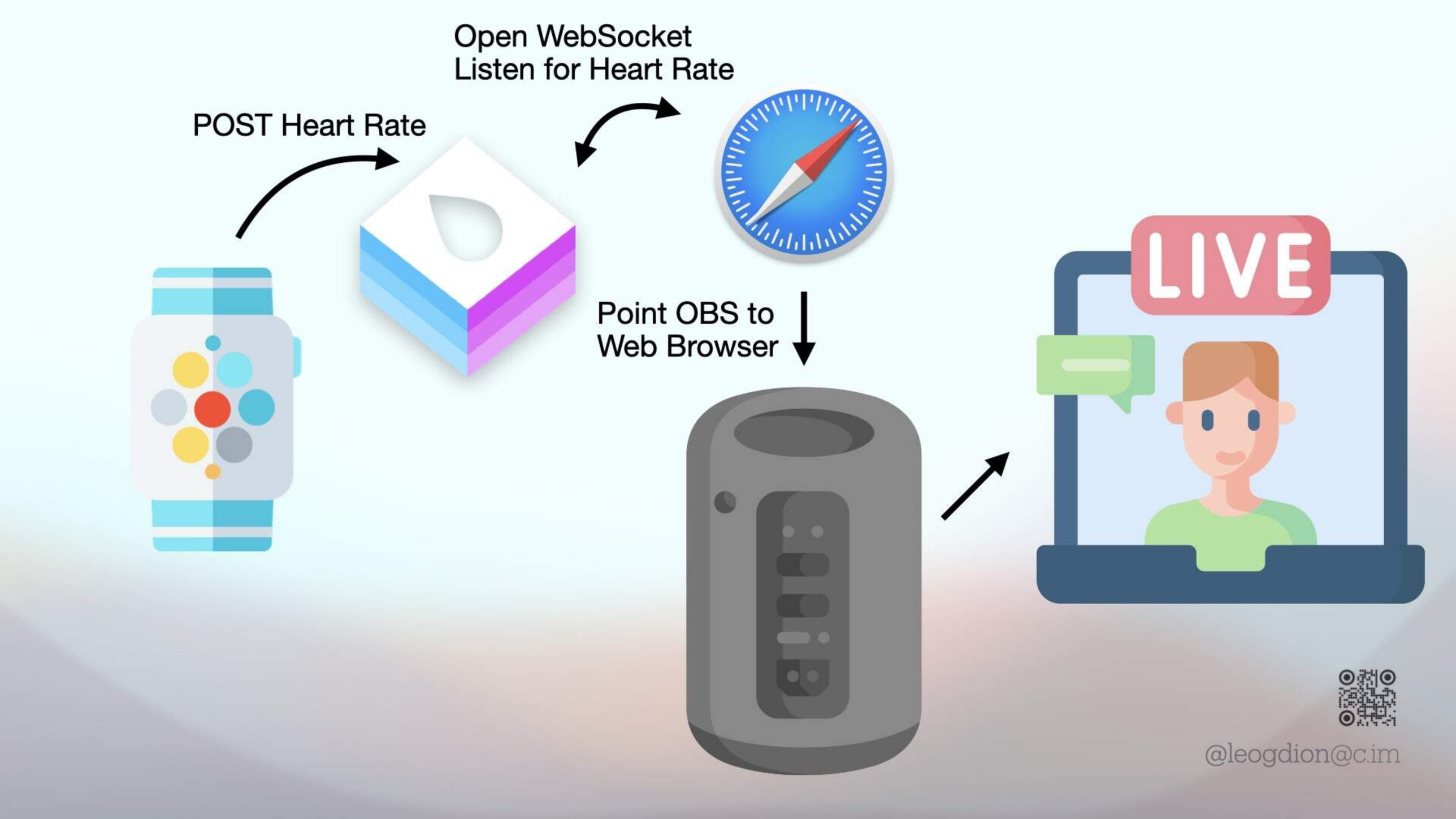

Private Database
Public Database

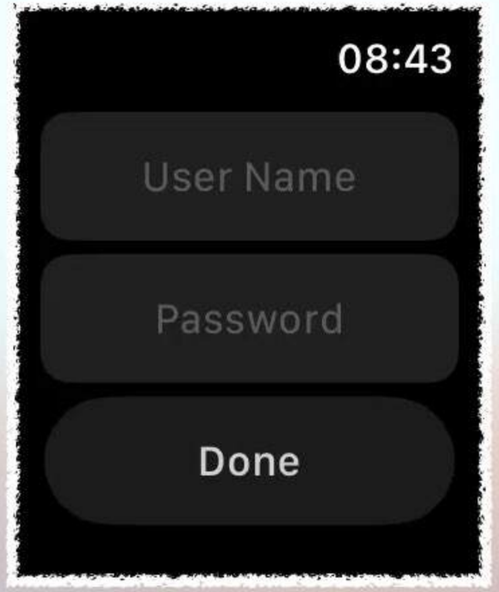
Private Database

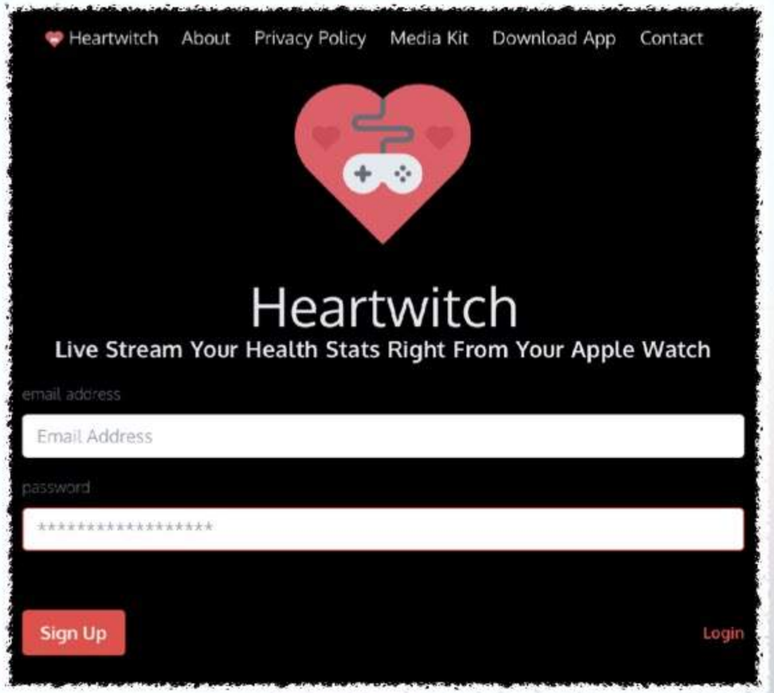
@leogdion@c.im

Heartwich

@leogdion@c.im

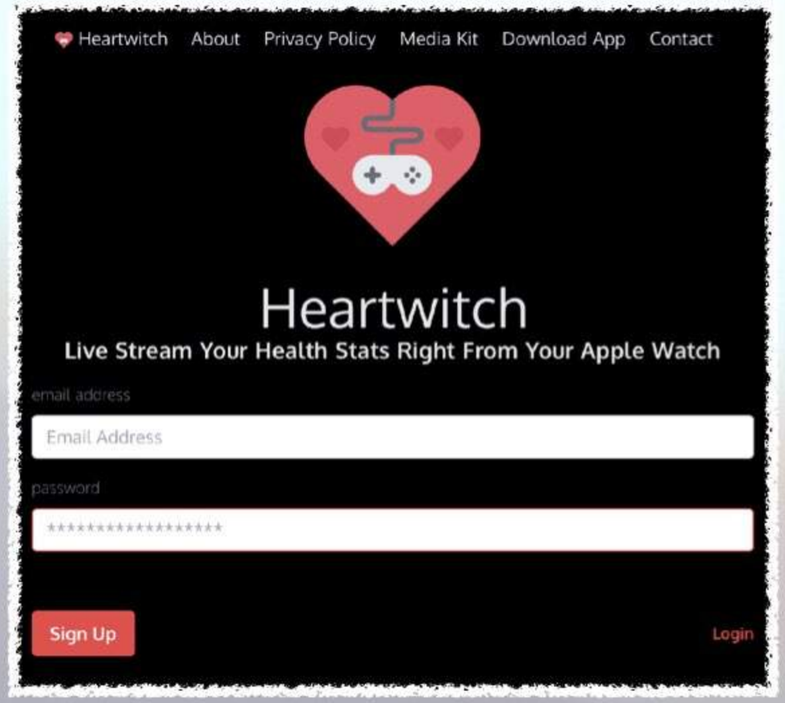

Open WebSocket
Listen for Heart Rate
POST Heart Rate
Point OBS to
Web Browser
LIVE
@leogdion@c.im

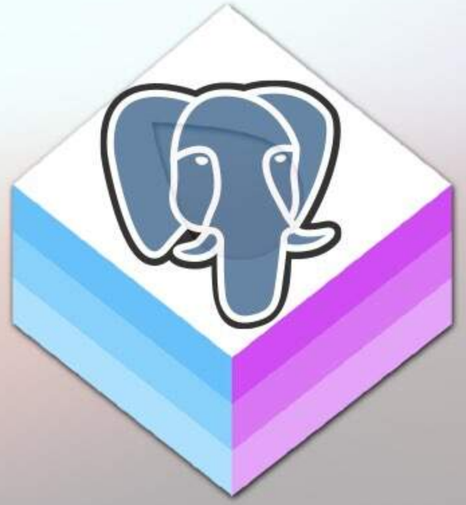




@leogdion@c.im

Leo's Apple Watch

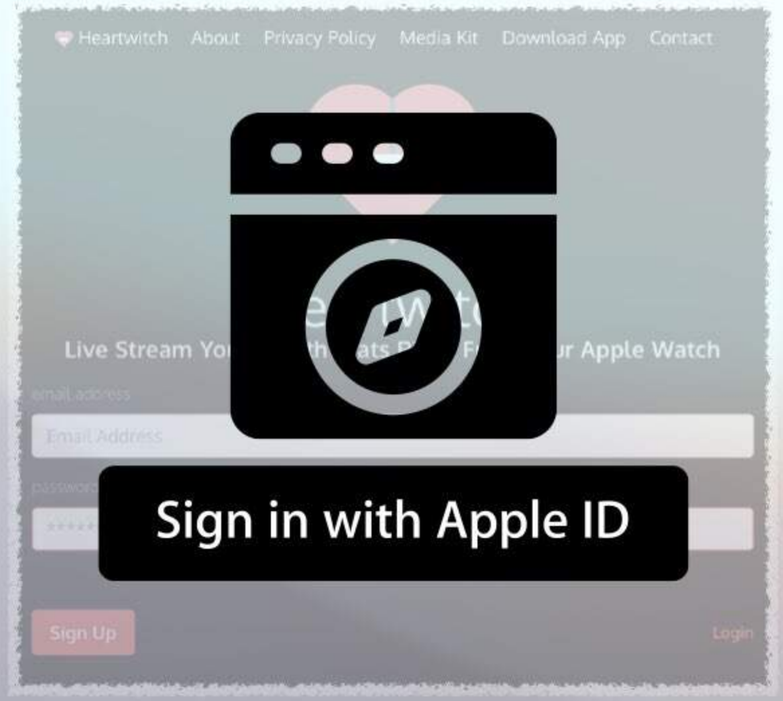


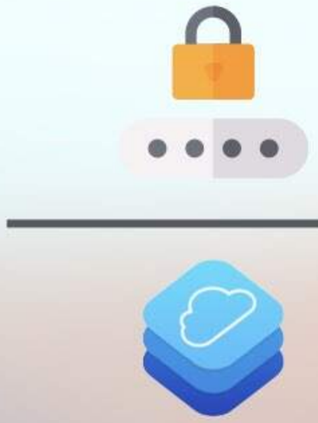


@leogdion@c.im

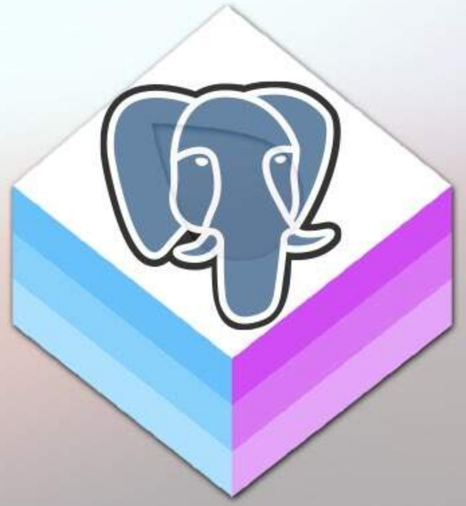
Leo's Apple Watch

Leo's Apple Watch



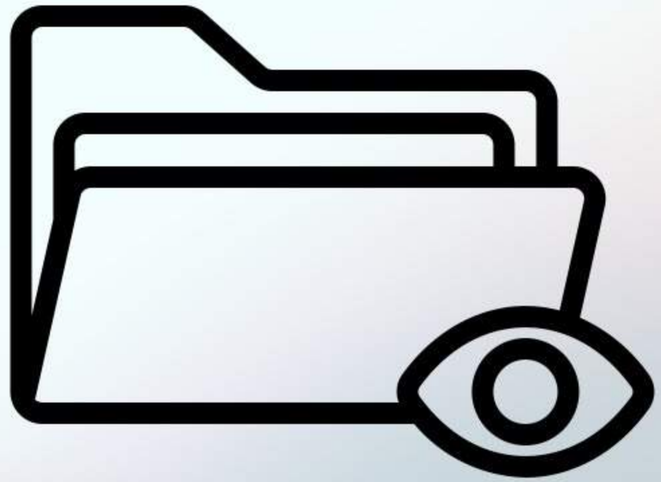


Web Auth Token



@leogdion@c.im

Public Database



@leogdion@c.im

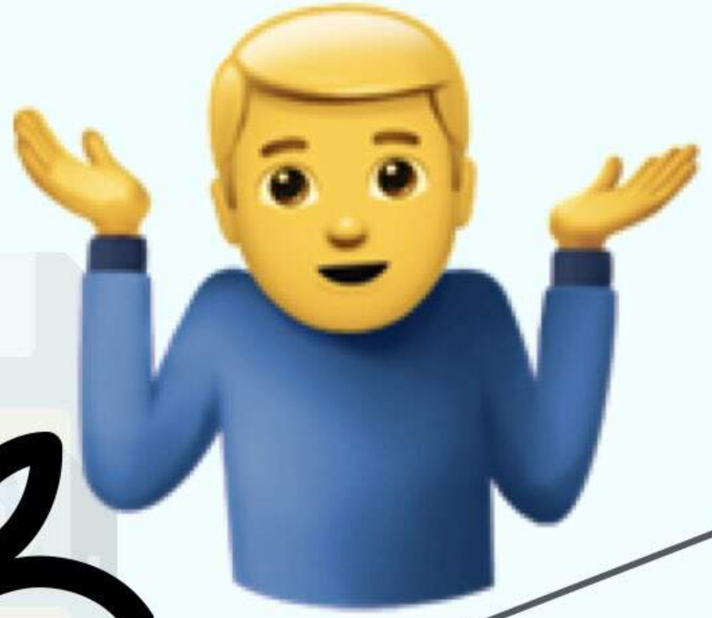
Bushel

@leogdion@c.im

Download Restore Image

Install Restore Image

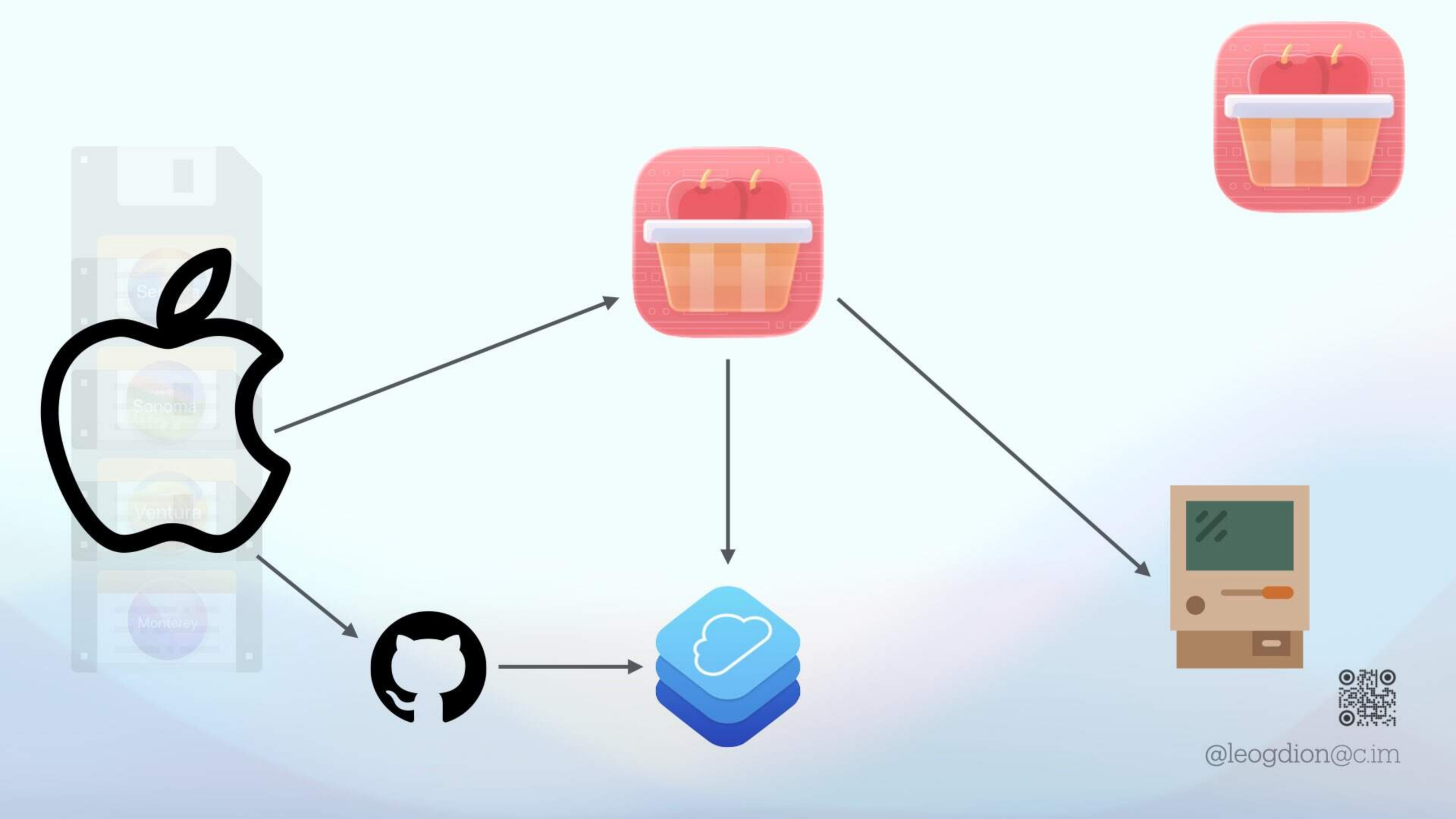
@leogdion@c.im



Download Restore Image

Install Restore Image

@leogdion@c.im


@leogdion@c.im

Building MistKit

https://github.com/brightdigit/MistKit

@leogdion@c.im

MistKit Web Demo

Authenticate with your Apple ID, then exercise the v1.0.0-beta.2 CloudKit surface through MistKit (server) or CloudKit JS (browser) and compare the wire-level behavior side-by-side.

Backend

MistKit (server) | CloudKit JS (browser)

MistKit mode routes browser → Hummingbird → CloudKit Web Services. CloudKit JS mode routes browser → CloudKit Web Services directly. Both share the same Apple ID session token, hit the same container, and exercise the same REST surface — only the SDK shape differs.

Database

Private | Public

Private uses the captured Apple ID web-auth token; Public uses server-to-server signing on the MistKit side and the API token on the CloudKit JS side.

Auth

Sign Out

Authenticated as _aca0fa3547ae9f9cd1f7e25fed948a20.

Notes

MistKit · Private · `records/query · records/modify`

Record type: Note

Limit: 20

Zone: zone for query & writes

Zone owner: owner (shared zones; used

Refresh

TITLE	INDEX	CREATED ↓	MODIFIED	
notification probe 0d3eb96e-258d-46c0-8f64-483d9bc9c447 You		9/4/26, 7:58 PM	9/4/26, 7:58 PM	Delete
Generate test You		9/4/26, 3:14 PM	9/4/26, 3:14 PM	Delete
Hello again You		9/2/26, 7:50 PM	9/2/26, 7:50 PM	Delete
notification probe 0fa461eb-a60c-4c5e-8f2a-e7961c5bf944 You		9/1/26, 12:08 AM	9/1/26, 12:08 AM	Delete
notification probe bf426266-a978-419d-b78d-e58c6131c51b You		9/1/26, 12:04 AM	9/1/26, 12:04 AM	Delete
notification probe 43f835fd-98ee-4731-894d-92d2509c82ac You		9/1/26, 12:02 AM	9/1/26, 12:02 AM	Delete
notification probe 1a161705-9340-		9/1/26, 12:01 AM	9/1/26, 12:01 AM	Delete

New note

Title: Hello from MistKit

Index: 0

Image `assets/upload`

Generate | Clear | No image attached.

Create | New | Delete

▶ Last raw response

Rereference asset `assets/rereference`

Reuse another note's `image` on a target note — no re-upload. Click a row to prefill the source.

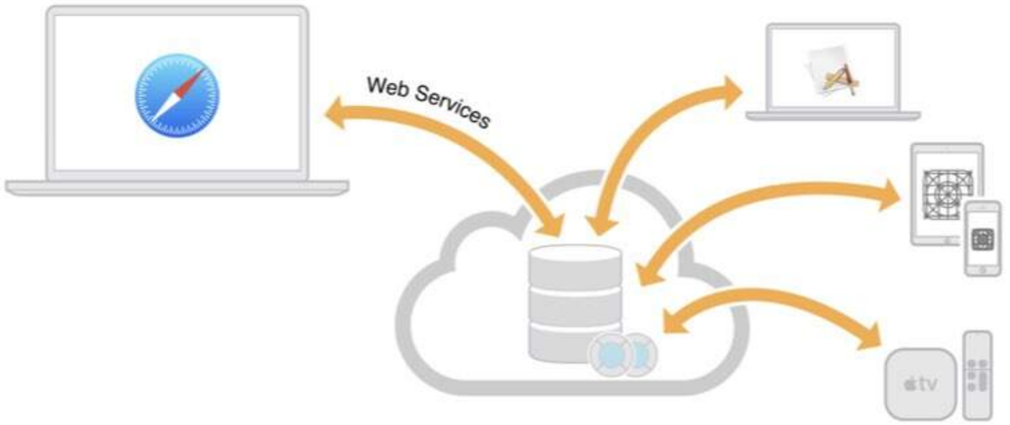
source record name | target record name

CloudKit Web Services

@leogdion@c.im

About CloudKit Web Services

You use the CloudKit native framework to take your app's existing data and store it in the cloud so that the user can access it on multiple devices. Then you can use CloudKit web services or CloudKit JS to provide a web interface for users to access the same data as your app. To use CloudKit web services, you must have the schema for your databases already created. CloudKit web services provides an HTTP interface to fetch, create, update, and delete records, zones, and subscriptions. You also have access to discoverable users and contacts. Alternatively, you can use the JavaScript API to access these services from a web app.



This document assumes that you are already familiar with CloudKit and CloudKit Dashboard. The following resources provide more information about CloudKit:

- *CloudKit Quick Start* and *CloudKit Framework Reference* teach you how to create a CloudKit app and use CloudKit Dashboard.
- *CloudKit JavaScript Reference* describes an alternative JavaScript API for accessing your app's CloudKit databases from a web app.
- *CloudKit Catalog: An Introduction to CloudKit (Cocoa and JavaScript)* sample code demonstrates CloudKit web services and CloudKit JS. For the interactive hosted version of this sample, go to CloudKit Catalog.

Document Revision History

This table describes the changes to *CloudKit Web Services Reference*.

Date	Notes
2016-06-13	Added sharing support and deprecated some endpoints.
2016-02-04	Added "Accessing CloudKit Using a Server-to-Server Key" section.
2015-12-10	Applied minor edits throughout.
2015-11-05	Applied minor edits throughout.
2015-09-16	First revision that describes the CloudKit web services protocol.

Document Revision History

This table describes the changes to *CloudKit Web Services Reference*.

Date	Notes
2016-06-13	Added sharing support and deprecated some endpoints.
2016-02-04	Added "Accessing CloudKit Using a Server-to-Server Key" section.
2015-12-10	Applied minor edits throughout.
2015-11-05	Applied minor edits throughout.
2015-09-16	First revision that describes the CloudKit web services protocol.

On This Page

Uploading Assets (assets/upload)

Request tokens for asset fields in new or existing records used in another request to upload asset data.

Path

```
POST [path]/database/[version]/[container]/[environment]/[database]/assets/upload
```

Parameters

path
The URL to the CloudKit web service, which is `https://api.apple-cloudkit.com`.

version
The protocol version—currently, 1.

container
A unique identifier for the app's container. The container ID begins with `iCloud.`.

environment
The version of the app's container. Pass `development` to use the environment that is not accessible by apps available on the store. Pass `production` to use the environment that is accessible by development apps and apps available on the store.

database
The database to store the data within the container. Pass `public` to use the database that is accessible to all users of the app. Pass `private` to use the database that is visible only to the currently signed-in user.

Request

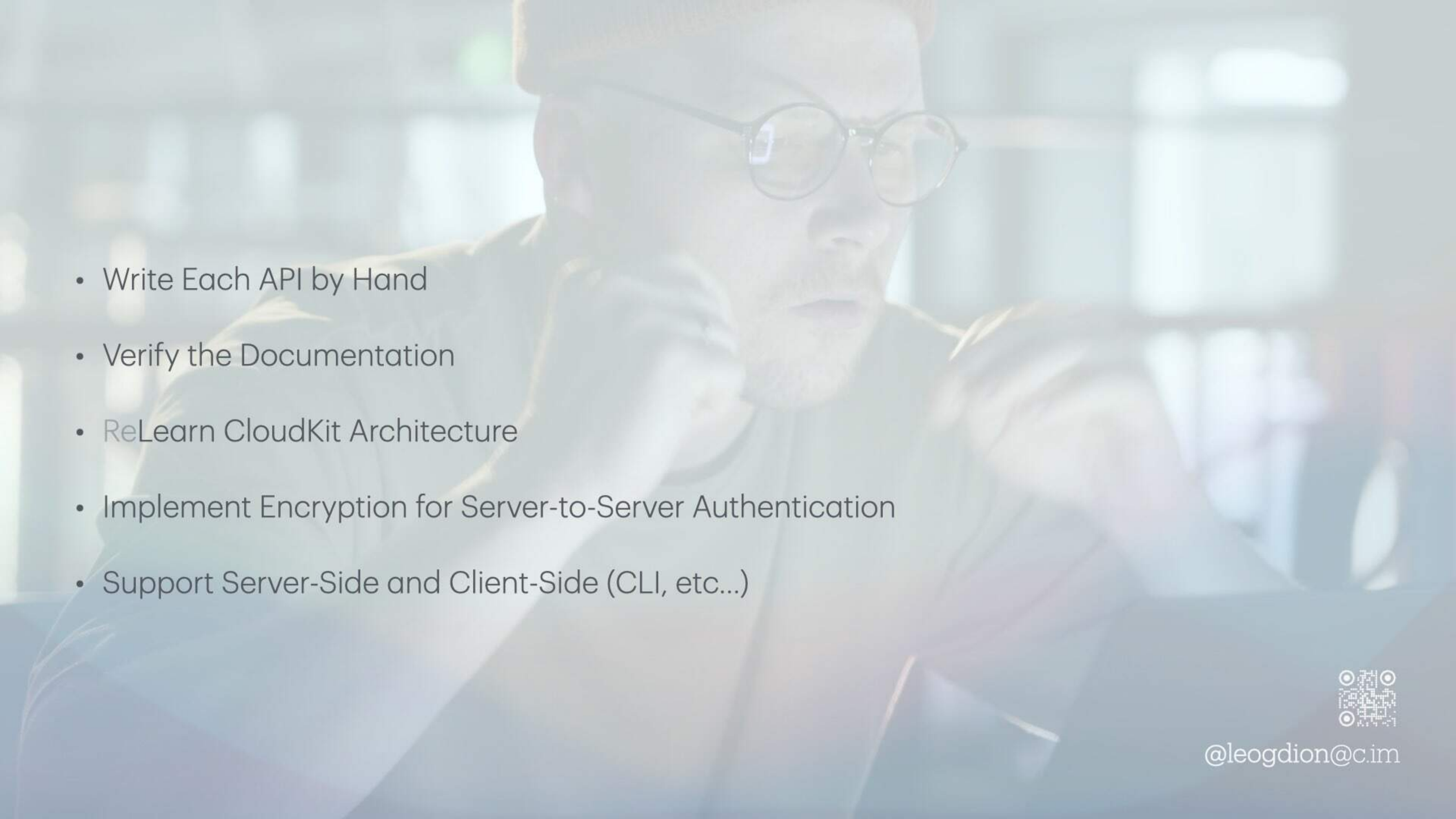
```
                    "receipt" : [RECEIPT],
                    "referenceChecksum" : [REFERENCE_CHECKSUM],
                    "size": [SIZE]
                }
            }
        },
        "recordName": "4c016e24-c397-4c9d-81d6-dee3e3a48bb0",
        "desiredKeys": []
    }
  }]
}'
```

If successful, the response is:

```
{
  "records": [{
      "recordName" : "4c016e24-c397-4c9d-81d6-dee3e3a48bb0",
      "recordChangeTag" : "1e",
      "created" : {
          "timestamp" : 1422312944104,
          "userRecordName" : "_9a065b60313a5937e75e03ed8e8f383d",
          "deviceID" : "iCloud"
      },
      "modified" : {
          "timestamp" : 1422312944104,
          "userRecordName" : "_9a065b60313a5937e75e03ed8e8f383d",
          "deviceID" : "iCloud"
      }
  }]
}
```

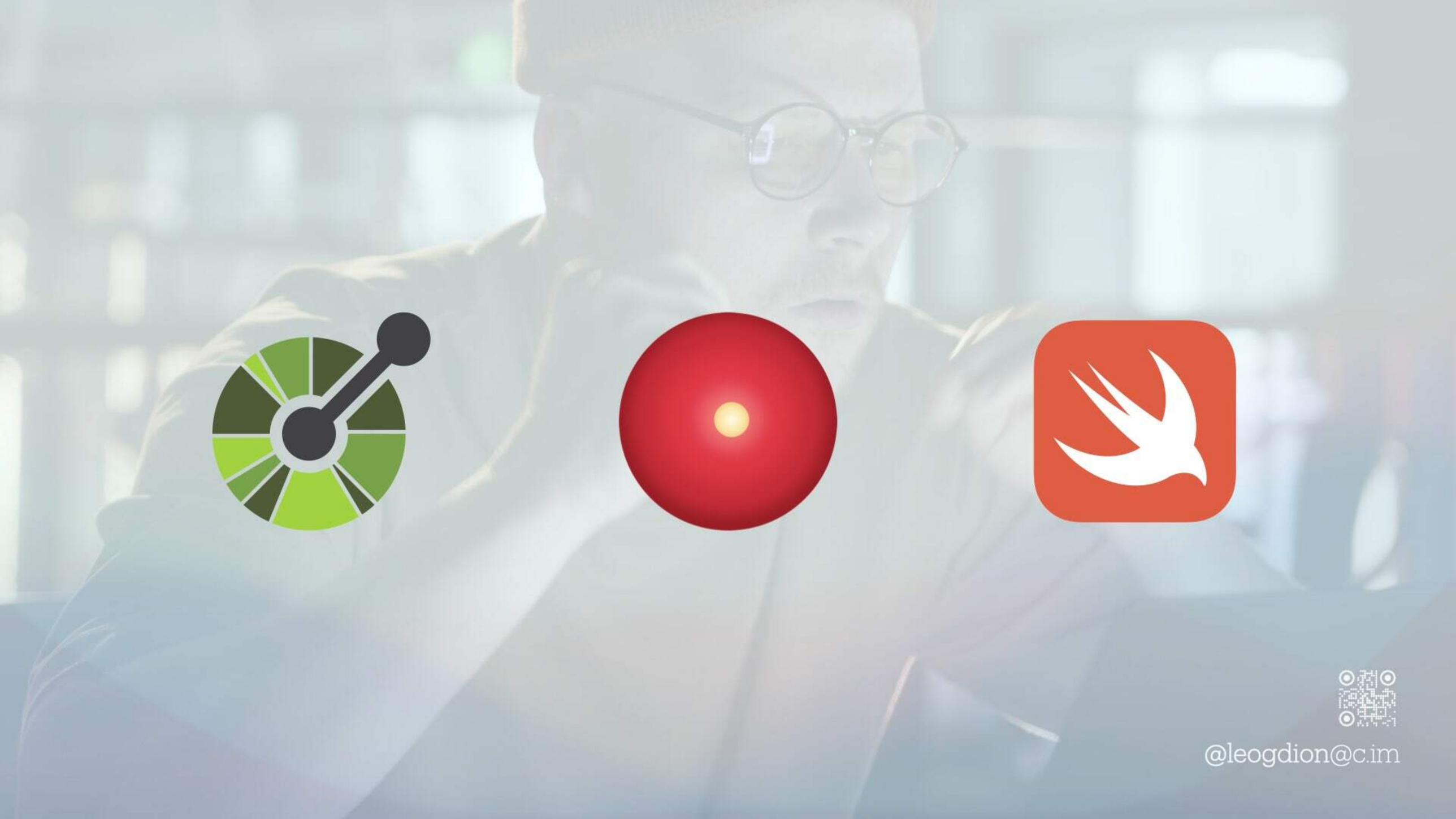
◀ Accepting Share Records (records/accept) Referencing Existing Assets (assets/rereference) ▶

@leogdion@c.im



- Write Each API by Hand
- Verify the Documentation
- ReLearn CloudKit Architecture
- Implement Encryption for Server-to-Server Authentication
- Support Server-Side and Client-Side (CLI, etc...)

@leogdion@c.im



@leogdion@c.im

@leogdion@c.im

OpenAPI Generator

https://github.com/apple/swift-openapi-generator

@leogdion@c.im

```yaml
openapi: 3.1.0

info:
  title: Todo API
  version: 1.0.0

servers:
  - url: https://api.example.com/v1

paths:
  /todos:
    get:
      summary: List all todos
      operationId: listTodos
      responses:
        "200":
          description: A list of todos
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: "#/components/schemas/Todo"

components:
  schemas:
    Todo:
      type: object
      required: [id, title, isCompleted]
      properties:
        id:
          type: string
        title:
          type: string
        isCompleted:
          type: boolean
```

OpenAPI

@leogdion@c.im

OpenAPI Generator

https://github.com/apple/swift-openapi-generator

@leogdion@c.im

README | Code of conduct | Contributing | Apache-2.0 license | Security

Swift OpenAPI Generator

sswg incubating | docc read documentation | release v1.13.1 | Swift 6.3 | 6.2 | 6.1

Generate Swift client and server code from an OpenAPI document.

Overview

OpenAPI is a specification for documenting HTTP services. An OpenAPI document is written in either YAML or JSON, and can be read by tools to help automate workflows, such as generating the necessary code to send and receive HTTP requests.

Swift OpenAPI Generator is a Swift package plugin that can generate the ceremony code required to make API calls, or implement API servers.

The code is generated at build-time, so it's always in sync with the OpenAPI document and doesn't need to be committed to your source repository.

Below you can see some example code, or you can follow one of the step-by-step tutorials.

Using a generated API client

The generated `Client` type provides a method for each HTTP operation defined in the OpenAPI document[1] and can be used with any HTTP library that provides an implementation of `ClientTransport`.

```
import OpenAPIURLSession
import Foundation

let client = Client(
    serverURL: URL(string: "http://localhost:8080/api")!,
    transport: URLSessionTransport()
)
let response = try await client.getGreeting()
print(try response.ok.body.json.message)
```

Using generated API server stubs

To implement a server, define a type that conforms to the generated `APIProtocol`, providing a method for each HTTP operation defined in the OpenAPI document[1].

The server can be used with any web framework that provides an implementation of `ServerTransport`, which allows you to register your API handlers with the HTTP server.

```
}
```

Package ecosystem

The Swift OpenAPI Generator project is split across multiple repositories to enable extensibility and minimize dependencies in your project.

Repository	Description
apple/swift-openapi-generator	Swift package plugin and CLI
apple/swift-openapi-runtime	Runtime library used by the generated code
apple/swift-openapi-urlsession	`ClientTransport` using URLSession
swift-server/swift-openapi-async-http-client	`ClientTransport` using AsyncHTTPClient
frameo-net/swift-okhttp	`ClientTransport` using OkHttp
vapor/swift-openapi-vapor	`ServerTransport` using Vapor
hummingbird-project/swift-openapi-hummingbird	`ServerTransport` using Hummingbird
awslabs/swift-openapi-lambda	`ServerTransport` using AWS Lambda

Swift OpenAPI Generator Tutorials

Working with Swift OpenAPI Generator

Learn how to use *Swift OpenAPI Generator* to build and maintain great HTTP services and clients that your adopters will love.

Spec-driven development eliminates the ambiguity of how your service behaves, and code-generation removes the margin for implementation error.

OpenAPI is an industry standard for declaring the functionality of your HTTP server and Swift OpenAPI Generator works seamlessly with Swift Package Manager (and Xcode) to make it easier than ever to build compliant servers and clients.

1hr 10min Estimated Time

Get started

@leogdion@c.im

Swift OpenAPI Generator

Generate Swift client and server code from an OpenAPI document.

Overview

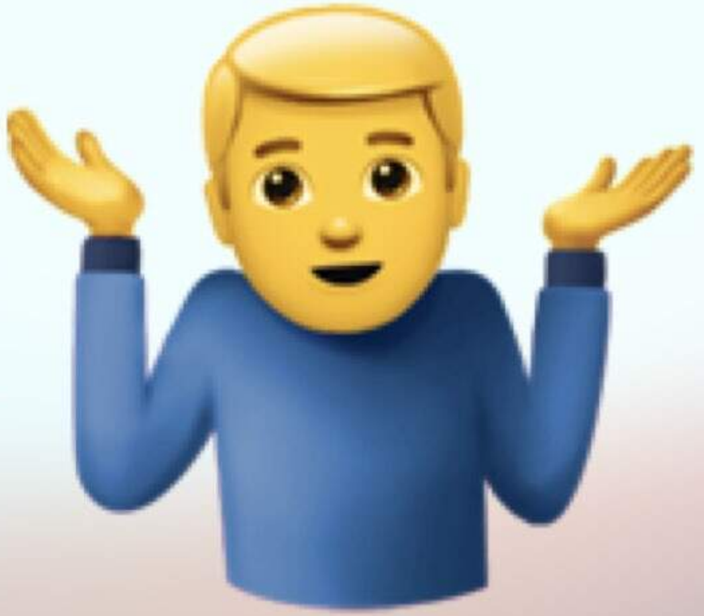
OpenAPI is a specification for documenting HTTP services. An OpenAPI document is written in either YAML or JSON, and can be read by tools to help automate workflows, such as generating the necessary code to send and receive HTTP requests.

Swift OpenAPI Generator is a Swift package plugin that can generate the ceremony code required to make API calls, or implement API servers.

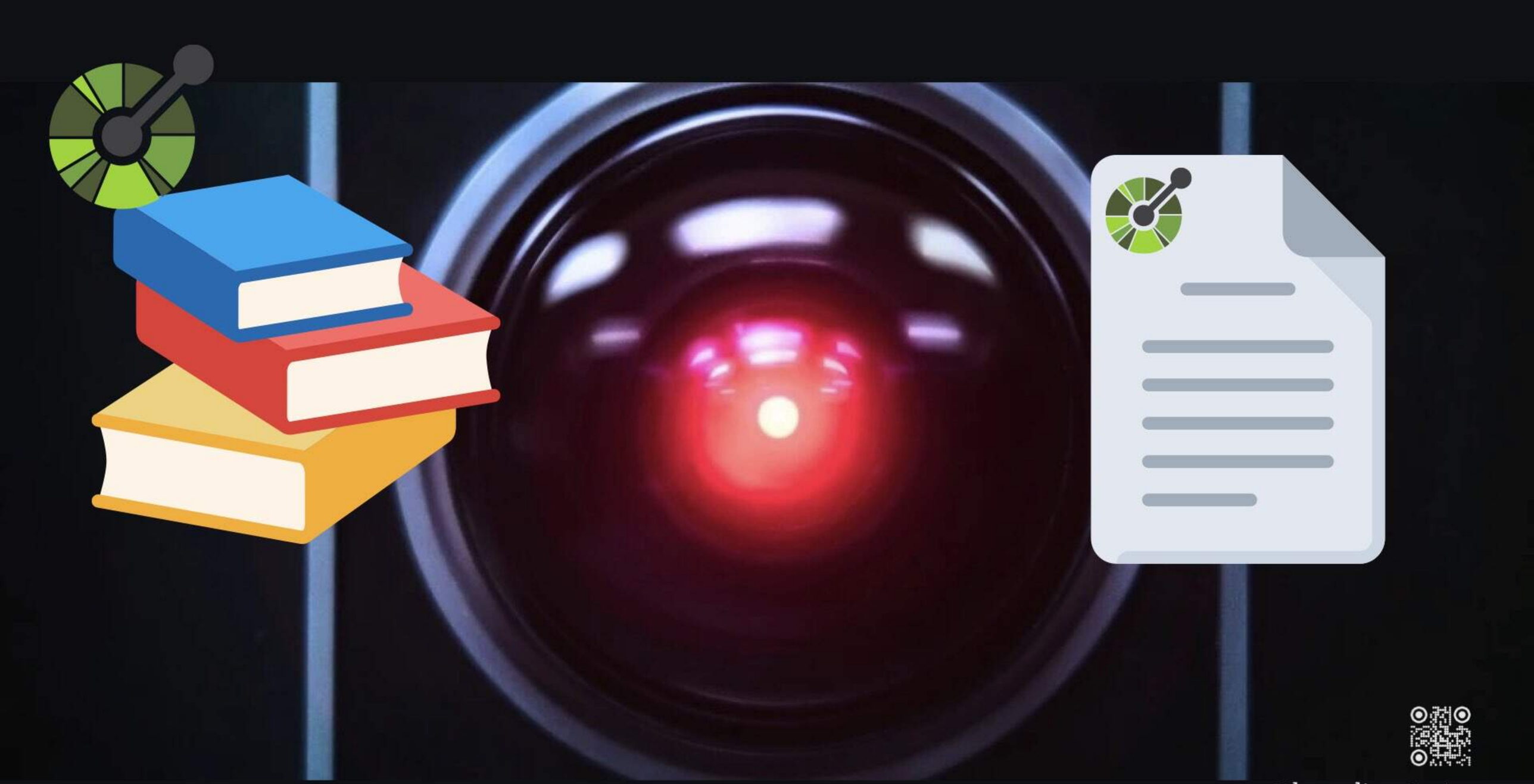
The code is generated at build-time, so it's always in sync with the OpenAPI document and doesn't need to be committed to your source repository.

Features

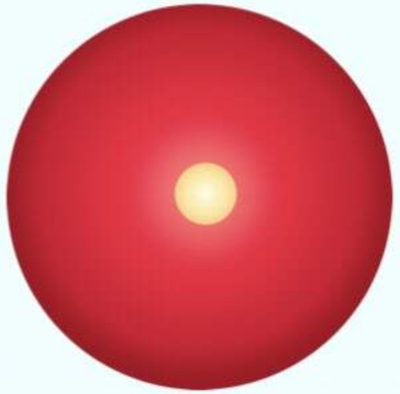
- Works with OpenAPI Specification versions 3.0 and 3.1 and has preliminary support for version 3.2.
- Streaming request and response bodies enabling use cases such as JSON event streams, and large payloads without buffering.



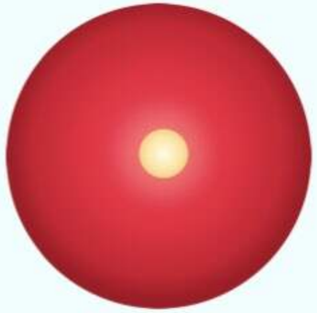
@leogdion@c.im



@leogdion@c.im



@leogdion@c.im

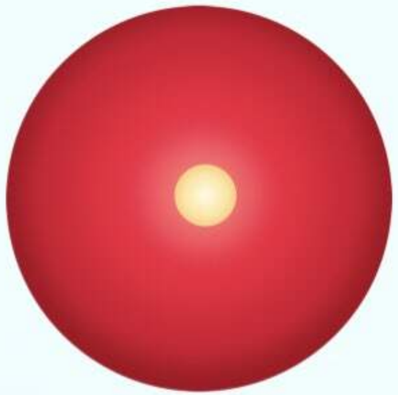


@leogdion@c.im

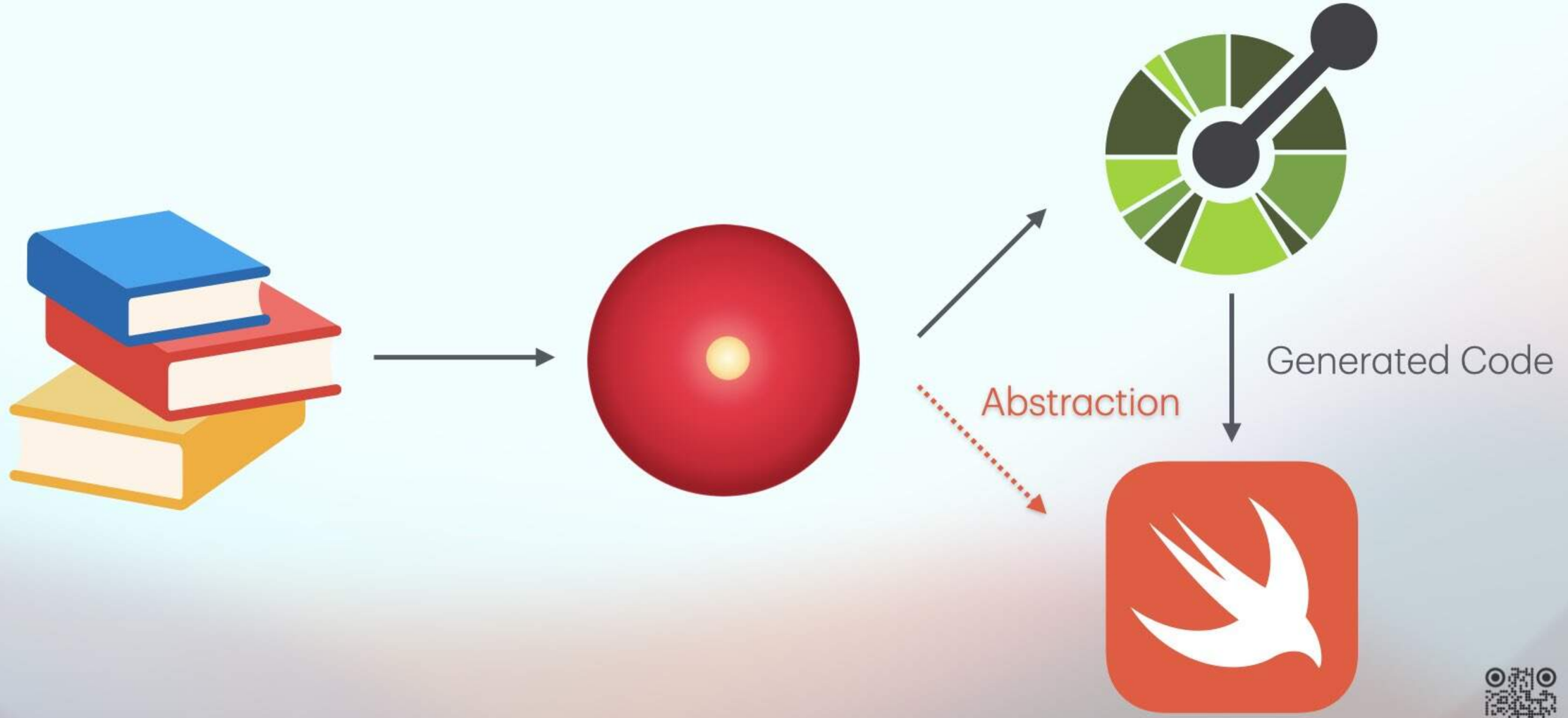
```yaml
openapi: 3.0.3
info:
  title: Apple CloudKit Web Services API
  description: |
    CloudKit web services provides an HTTP interface to fetch, create, update,
and delete records, zones, and subscriptions.
    You also have access to discoverable users and contacts.

    ## Authentication
    There are two authentication methods:
    1. API Token Authentication - Use query parameters: `?ckAPIToken=[API
token]&ckWebAuthToken=[Web Auth Token]`
    2. Server-to-Server Key Authentication - Pass the key ID as `X-Apple-
CloudKit-Request-KeyID` header

    ## Base URL Structure
    `https://api.apple-cloudkit.com/database/{version}/{container}/
{environment}/{database}/{operation}`

    Where:
    - version: Protocol version (currently "1")
    - container: Unique identifier for the app's container (begins with
"iCloud.")
```

```
        $ref: '#/components/responses/Failure'
      '503':
        $ref: '#/components/responses/Failure'

/database/{version}/{container}/{environment}/{database}/zones/lookup:
  post:
    summary: Lookup Zones
    description: Fetch specific zones by their IDs
    operationId: lookupZones
    tags:
      - Zones
    parameters:
      - $ref: '#/components/parameters/version'
      - $ref: '#/components/parameters/container'
      - $ref: '#/components/parameters/environment'
      - $ref: '#/components/parameters/database'
    requestBody:
      required: true
      content:
        application/json:
          schema:
            type: object
            properties:
              zones:
```

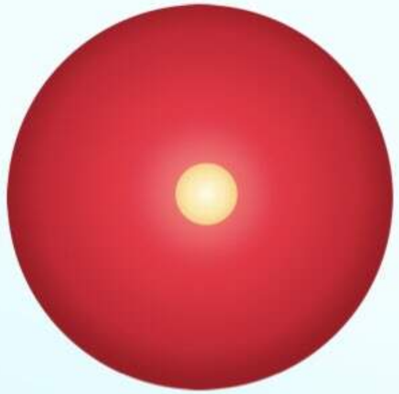
@leogdion@c.im



@leogdion@c.im

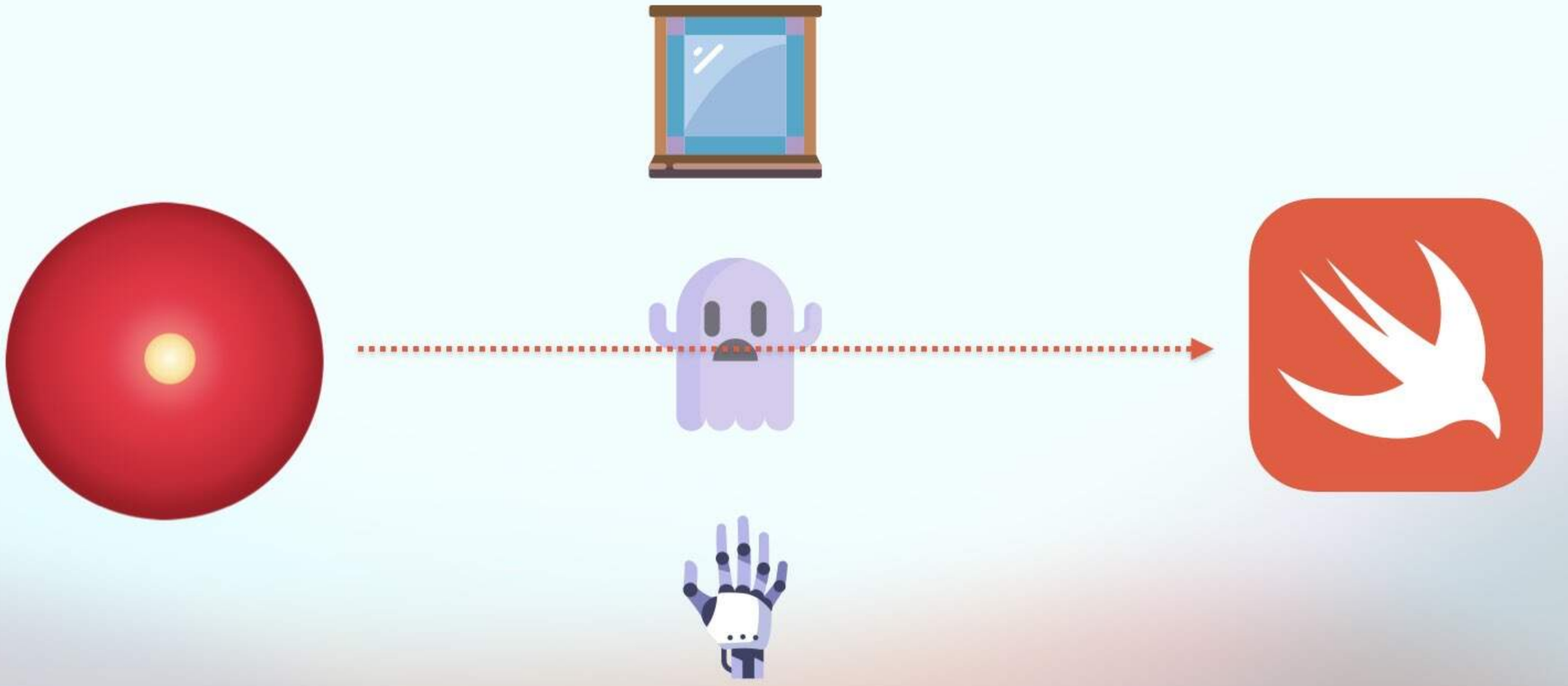


@leogdion@c.im



Abstraction

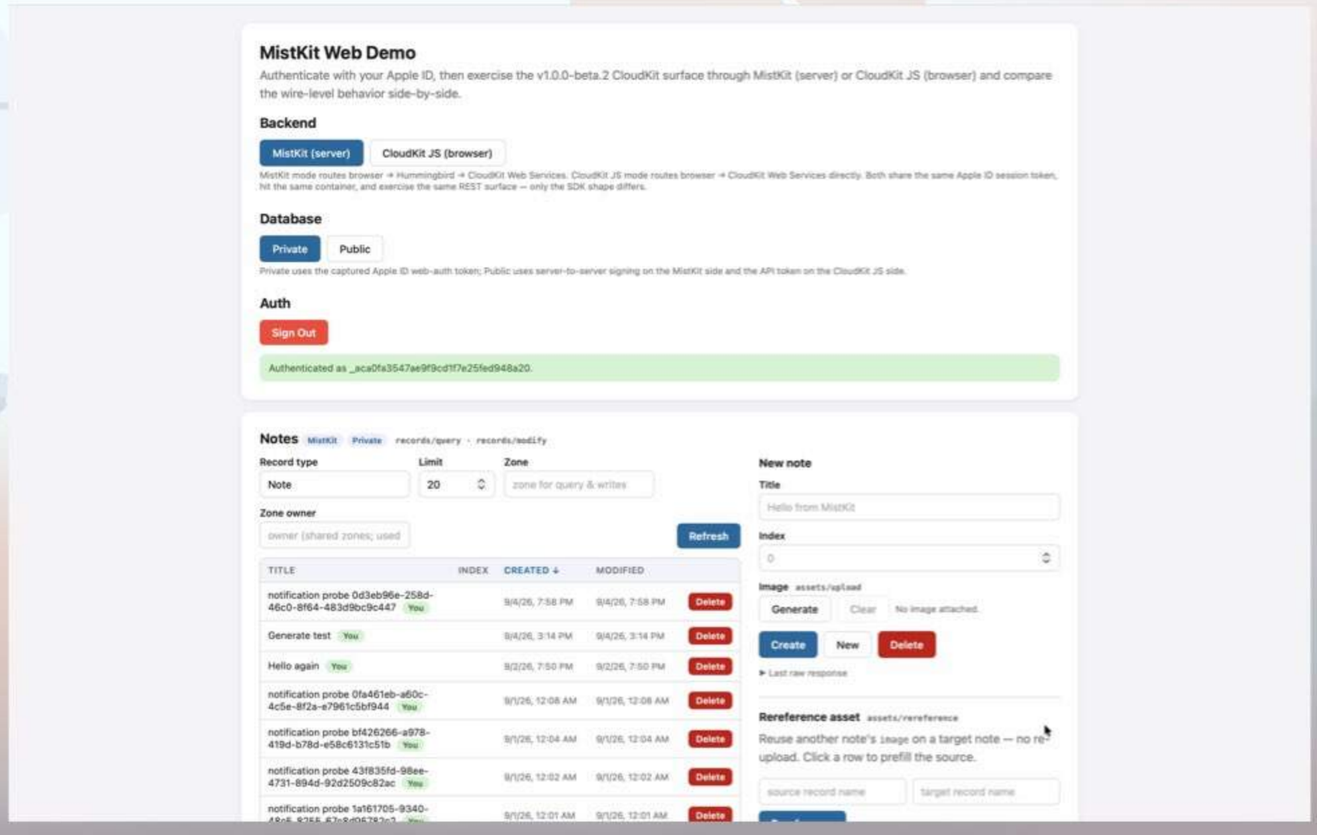
@leogdion@c.im

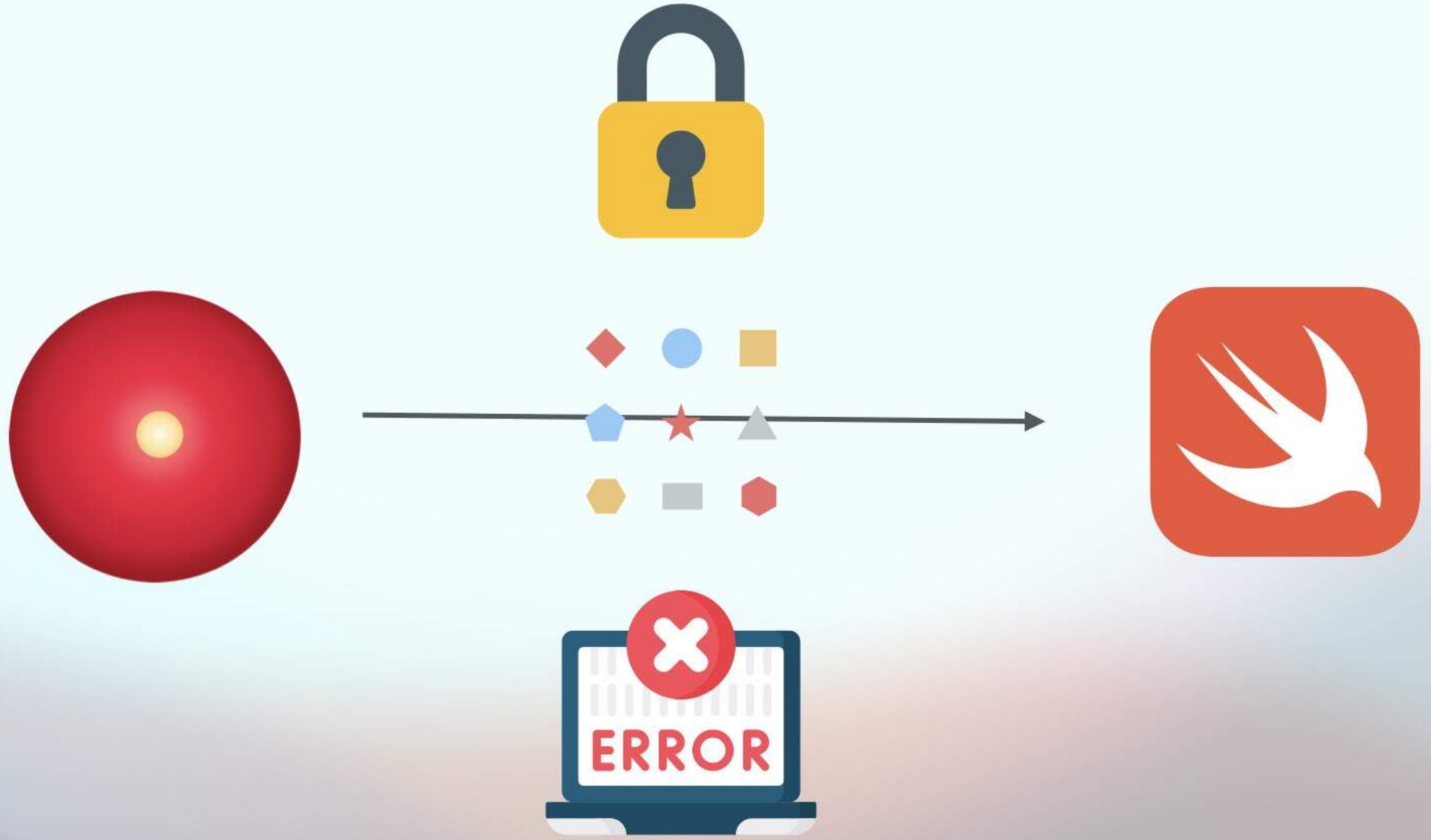


@leogdion@c.im

Integration Tests

User Testing





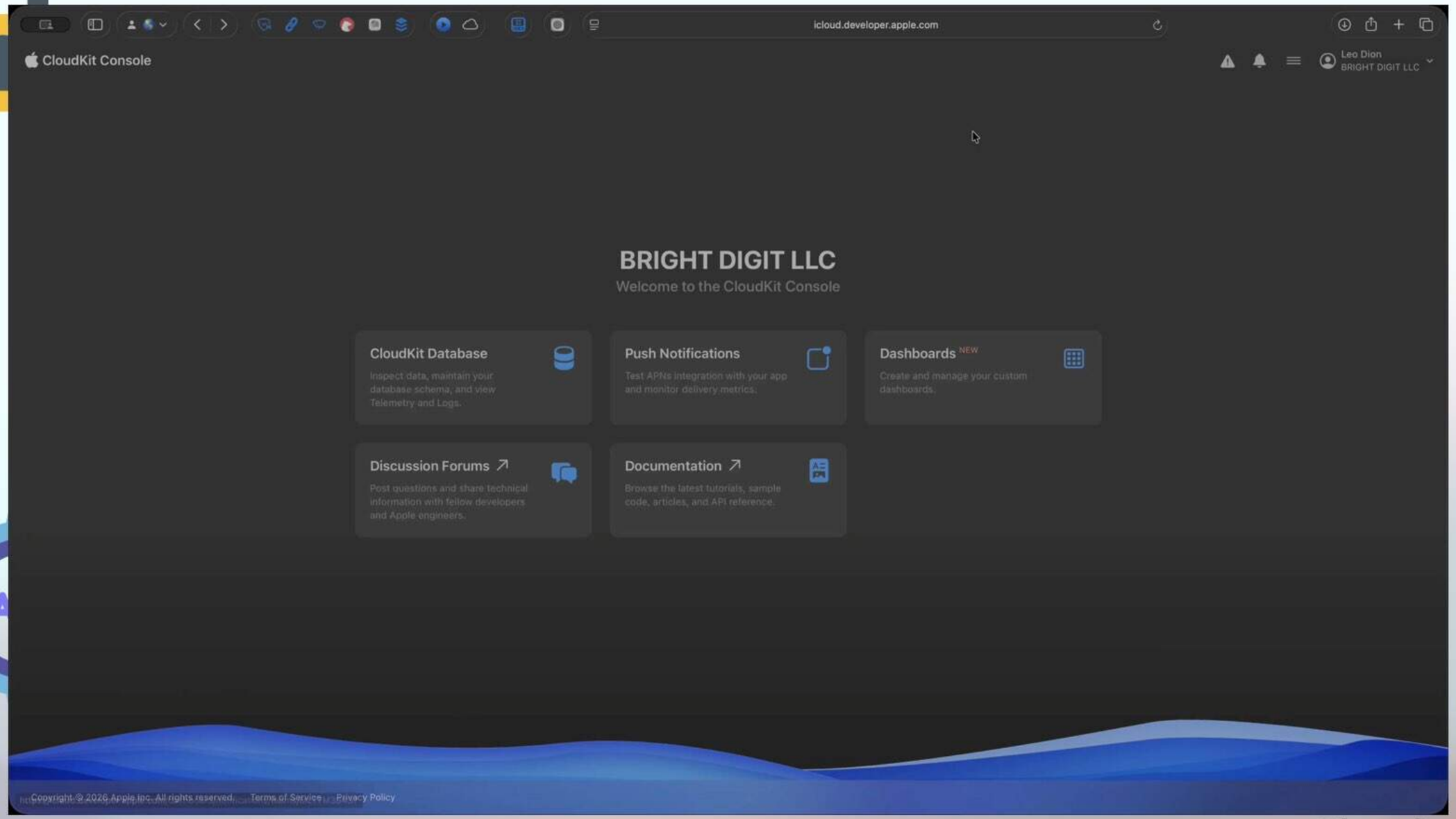
@leogdion@c.im

Authenticat hods

@leogdion@c.im

Authenticat hods

@leogdion@c.im



@leogdion@c.im

API Token

@leogdion@c.im

Monitor
Telemetry
Usage
Logs
Alerts

Data
Records
Zones
Subscriptions

Schema
Indexes – Modified
Record Types – Modified
Security Roles – Modified
History

Settings
Tokens & Keys
Sharing Fallback

Act As iCloud Account...
Deploy Schema Changes...
Import Schema...
Export Schema...
Reset Environment...
Container Permissions

API Tokens

API Tokens allow a web frontend to send requests to this container and environment.

Name
test
Vapor Token
MistKit Demo
Token

Server-to-Server Keys

A server-to-server key allows a server-side application to read and write data in the public database of this container and environment.

Name
26.04.14
Demo User
Another Demo

@leogdion@c.im

```
?ckAPIToken=[API token]
```

@leogdion@c.im

```
?ckAPIToken=[API token]
```


```
CloudKit.configure({
    containers: [{
        containerIdentifier: '[insert your container ID here]',
        apiTokenAuth: {
            apiToken: '[insert your API token]'
        },
        environment: 'development'
    }]
});
```

@leogdion@c.im

Web Authentication

@leogdion@c.im

Monitor
Telemetry
Usage
Logs
Alerts

Data
Records
Zones
Subscriptions

Schema
Indexes – Modified
Record Types – Modified
Security Roles – Modified
History

Settings
Tokens & Keys
Sharing Fallback

Act As iCloud Account...
Deploy Schema Changes...
Import Schema...
Export Schema...
Reset Environment...
Container Permissions

Tokens & Keys

"Demo Presentation " Details

··· Cancel Save Changes

API Access 3537be7f4545f9b2644f2097c23b725d17a2c07e5c8be147cd66ef17bc4cfda5

Name Demo Presentation

Sign in Callback

- (•) Post Message
  Allows the Apple Account sign in browser window to communicate the authentication result back to your web application using the CloudKit JS postMessage API.
- ( ) URL Redirect
  Redirects the browser back to this URL after the Apple Acount sign in has completed.

https:// Enter a custom URL...

Allowed Origins

- (•) Any Domain
- ( ) Only the following domain(s):

Discoverability

- [ ] Request user discoverability at sign in.

Notes Notes about this API key

@leogdion@c.im

Tokens & Keys

“Demo Presentation ” Details

API Access

3537be7f4545f9b2644f2097c23b725d17a2c07e5c8be147cd66ef17bc4cfda5

Name

Demo Presentation

Sign in Callback

- (•) Post Message
  Allows the Apple Account sign in browser window to communicate the authentication result back to your web application using the CloudKit JS postMessage API.
- ( ) URL Redirect
  Redirects the browser back to this URL after the Apple Acount sign in has completed.

https:// ⌄ | Enter a custom URL...

Allowed Origins

- (•) Any Domain
- ( ) Only the following domain(s):

Discoverability

- [ ] Request user discoverability at sign in.

Notes

Notes about this API key

Web Authentication

```
?ckAPIToken=[API token]
```

@leogdion@c.im

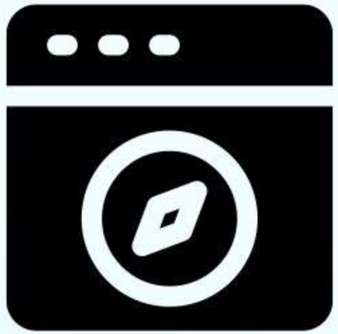
Web Authentication

```
?ckAPIToken=[API token]&ckWebAuthToken=[Web Auth Token]
```

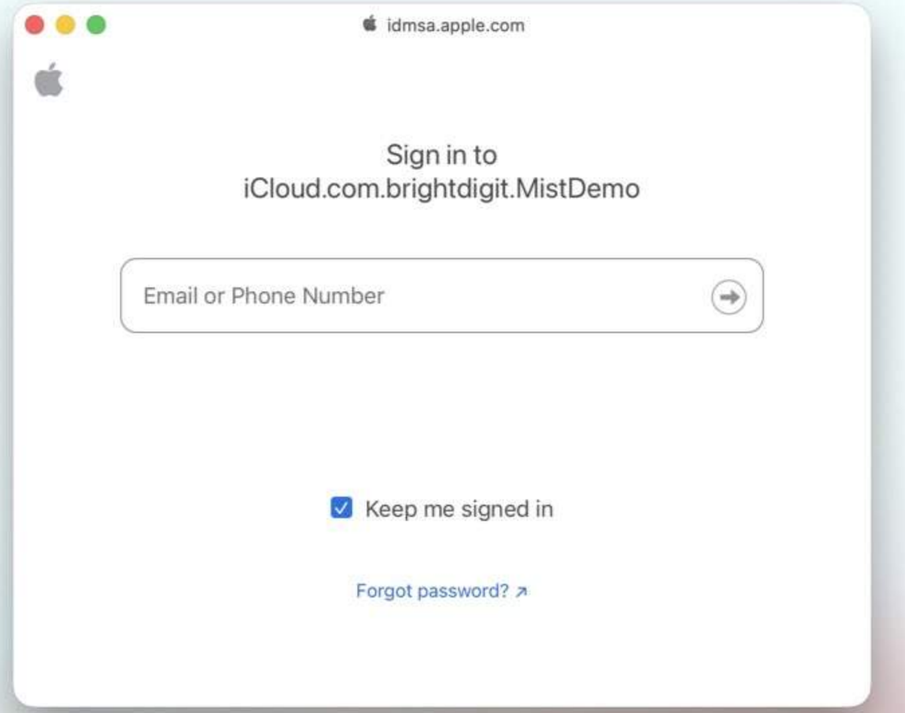
@leogdion@c.im

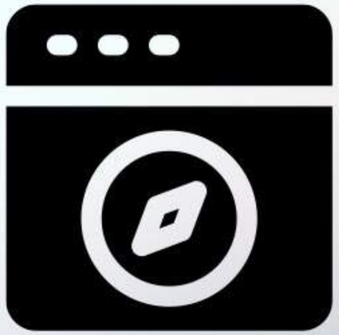
```
window.addEventListener('message', function(e) {
    console.log(e.data.ckWebAuthToken);
})
```

Post Message



Sign in with Apple ID

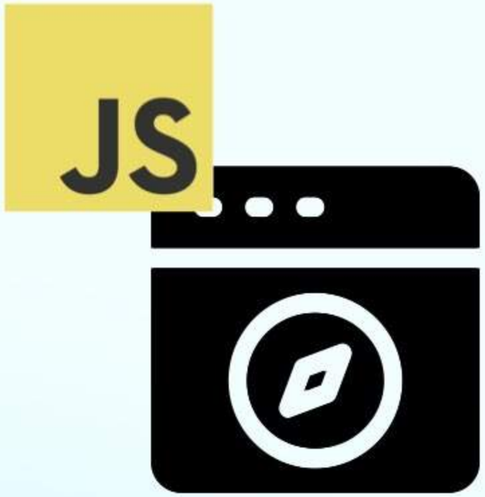




https://[callback-url]/?
ckWebAuthToken=[Web Auth Token]

URL Callback

@leogdion@c.im



@leogdion@c.im

CKFetchWebAuthTokenOperation

@leogdion@c.im

CKFetchWebAuthTokenOperation

```
let operation = CKFetchWebAuthTokenOperation(apiToken: apiToken)
operation.qualityOfService = .userInitiated
operation.fetchWebAuthTokenResultBlock = { @Sendable result in
  // do something with the result
}
add(operation)
```

@leogdion@c.im

CKFetchWebAuthTokenOperation

```
extension CKDatabase {
  internal func fetchWebAuthToken(apiToken: String) async throws
  -> String {
    try await withCheckedThrowingContinuation { continuation in
      let operation = CKFetchWebAuthTokenOperation(
        apiToken: apiToken
      )
      operation.qualityOfService = .userInitiated
      operation.fetchWebAuthTokenResultBlock = {
        @Sendable result in
        continuation.resume(with: result)
      }
      add(operation)
    }
  }
}
```

@leogdion@c.im

Web Authentication

```
?ckAPIToken=[API token]&ckWebAuthToken=[Web Auth Token]
```

@leogdion@c.im

Server to Server

@leogdion@c.im

Monitor
Telemetry
Usage
Logs
Alerts

Data
Records
Zones
Subscriptions

Schema
Indexes – Modified
Record Types – Modified
Security Roles – Modified
History

Settings
Tokens & Keys
Sharing Fallback

Act As iCloud Account...
Deploy Schema Changes...
Import Schema...
Export Schema...
Reset Environment...
Container Permissions

Tokens & Keys

New Server-to-Server Key

Cancel Save

Key ID

Name

Enter a name for the server-to-server key

Public Key

Step 1. Generate a private key in Terminal using:

```
openssl ecparam -name prime256v1 -genkey -noout -out eckey.pem
```

Note: Never expose or share the above private key with anyone, including Apple, as it has unrestricted access to your public database.

Step 2. Output a public key to be stored with the CloudKit server:

```
openssl ec -in eckey.pem -pubout
```

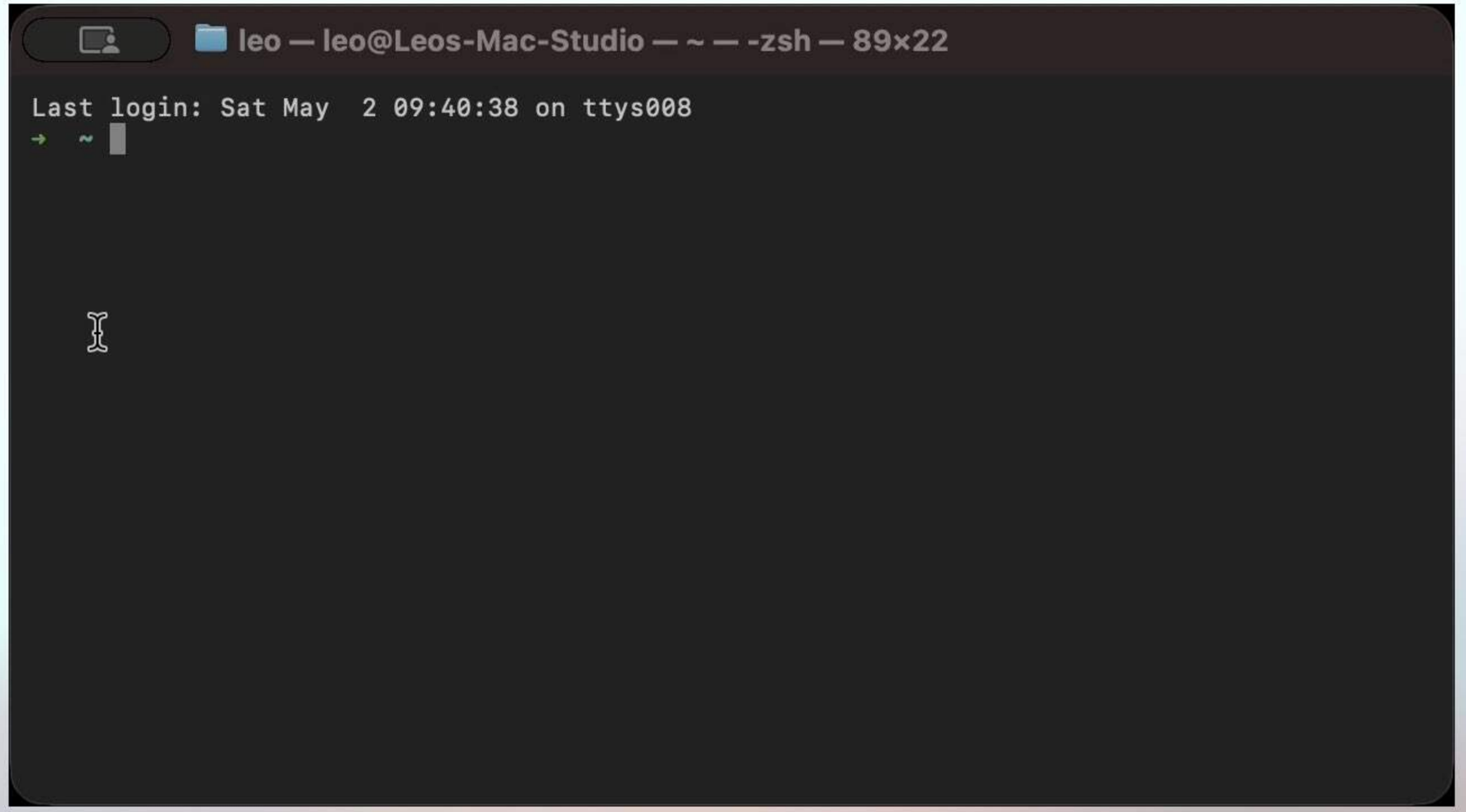
Step 3. Copy and paste your public key here (only the value between the BEGIN and END text):

Paste your public key here...

Notes

Notes about this server-to-server key

@leogdion@c.im



@leogdion@c.im

Monitor
Telemetry
Usage
Logs
Alerts

Data
Records
Zones
Subscriptions

Schema
Indexes – Modified
Record Types – Modified
Security Roles – Modified
History

Settings
Tokens & Keys
Sharing Fallback

Act As iCloud Account...
Deploy Schema Changes...
Import Schema...
Export Schema...
Reset Environment...
Container Permissions

Tokens & Keys

New Server-to-Server Key

Cancel Save

Key ID

Name

Enter a name for the server-to-server key

Public Key

Step 1. Generate a private key in Terminal using:

```
openssl ecparam -name prime256v1 -genkey -noout -out eckey.pem
```

Note: Never expose or share the above private key with anyone, including Apple, as it has unrestricted access to your public database.

Step 2. Output a public key to be stored with the CloudKit server:

```
openssl ec -in eckey.pem -pubout
```

Step 3. Copy and paste your public key here (only the value between the BEGIN and END text):

Paste your public key here...

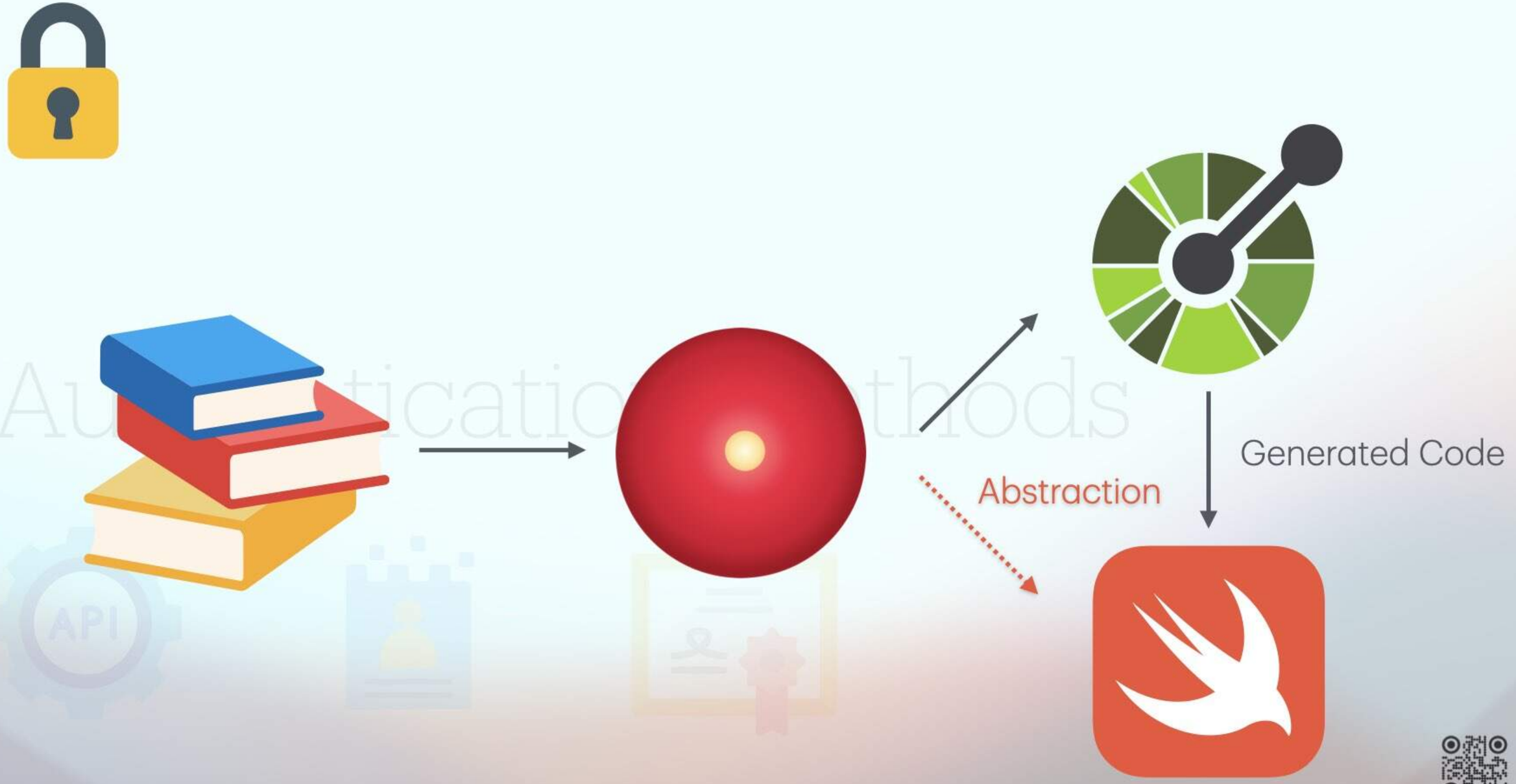
Notes

Notes about this server-to-server key

@leogdion@c.im

Authentication Methods

@leogdion@c.im



@leogdion@c.im

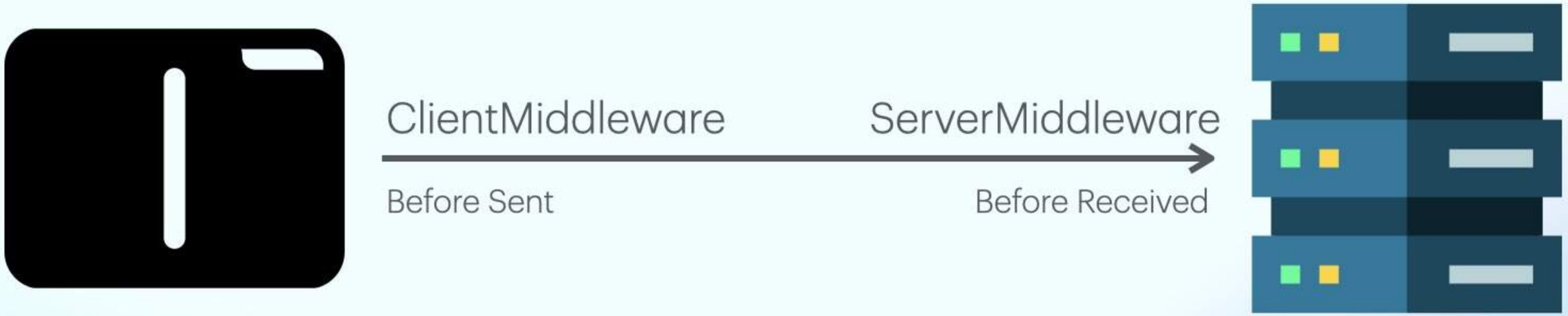
OpenAPI Middleware

@leogdion@c.im

OpenAPI Middleware

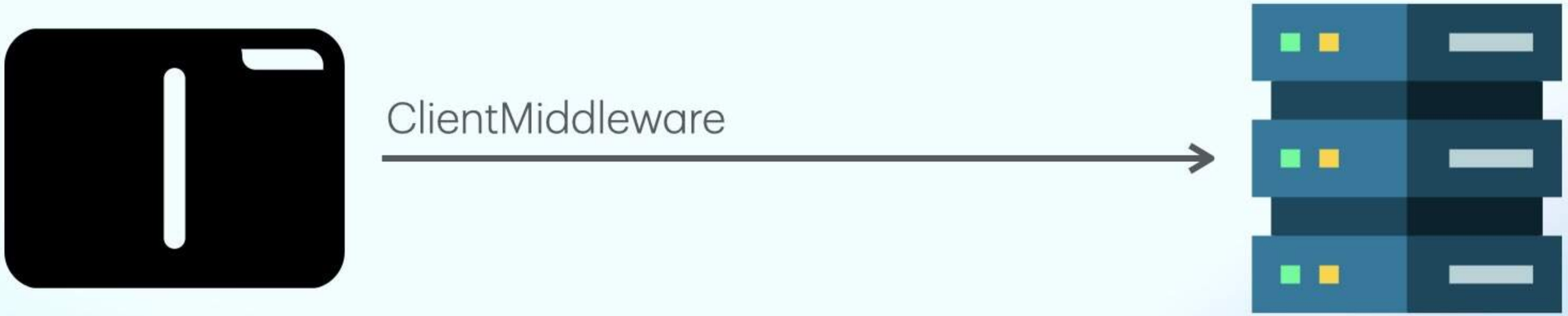
@leogdion@c.im

OpenAPI Middleware



@leogdion@c.im

OpenAPI Middleware



Protocol

ClientMiddleware

A type that intercepts HTTP requests and responses.

```
protocol ClientMiddleware : Sendable
```

ClientTransport.swift

Overview

It allows you to read and modify the request before it is received by the transport and the response after it is returned by the transport.

Appropriate for handling authentication, logging, metrics, tracing, injecting custom headers such as “user-agent”, and more.

Implement a custom client middleware

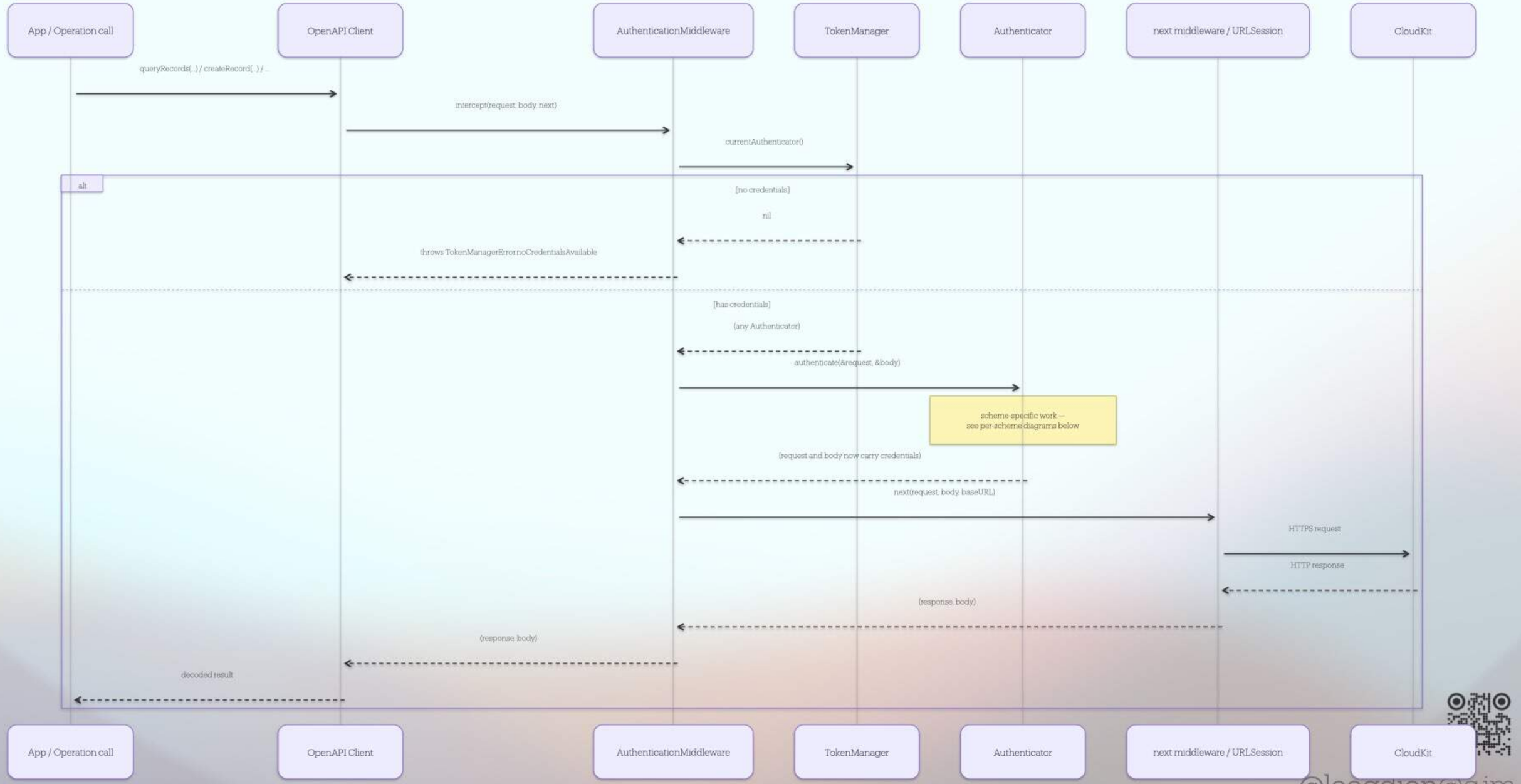
If a client middleware implementation with your desired behavior doesn't yet exist, or you need to simulate rare network conditions your tests, consider implementing a custom client middleware.

For example, to implement a middleware that injects the "Authorization" header to every outgoing request, define a new struct that conforms to the `ClientMiddleware` protocol:

```
/// Injects an authorization header to every request.
struct AuthenticationMiddleware: ClientMiddleware {

    /// The token value.
    var bearerToken: String

    func intercept(
        _ request: HTTPRequest,
        body: HTTPBody?,
        baseURL: URL,
        operationID: String,
        next: (HTTPRequest, HTTPBody?, URL) async throws -> (HTTPResponse, HTTPBody?)
    ) async throws -> (HTTPResponse, HTTPBody?) {
        var request = request
        request.headerFields[.authorization] = "Bearer \(bearerToken)"
        return try await next(request, body, baseURL)
    }
}
```

```swift
internal struct AuthenticationMiddleware: ClientMiddleware {
  internal let tokenManager: any TokenManager

  internal func intercept(
    _ request: HTTPRequest,
    body: HTTPBody?,
    baseURL: URL,
    operationID: String,
    next: (HTTPRequest, HTTPBody?, URL) async throws -> (HTTPResponse, HTTPBody?)
  ) async throws -> (HTTPResponse, HTTPBody?) {
    guard let authenticator = try await tokenManager.currentAuthenticator() else {
      throw TokenManagerError.invalidCredentials(.noCredentialsAvailable)
    }

    var modifiedRequest = request
    var modifiedBody = body
    try await authenticator.authenticate(request: &modifiedRequest, body: &modifiedBody)
    return try await next(modifiedRequest, modifiedBody, baseURL)
  }
}
```

@leogdion@c.im



API Token

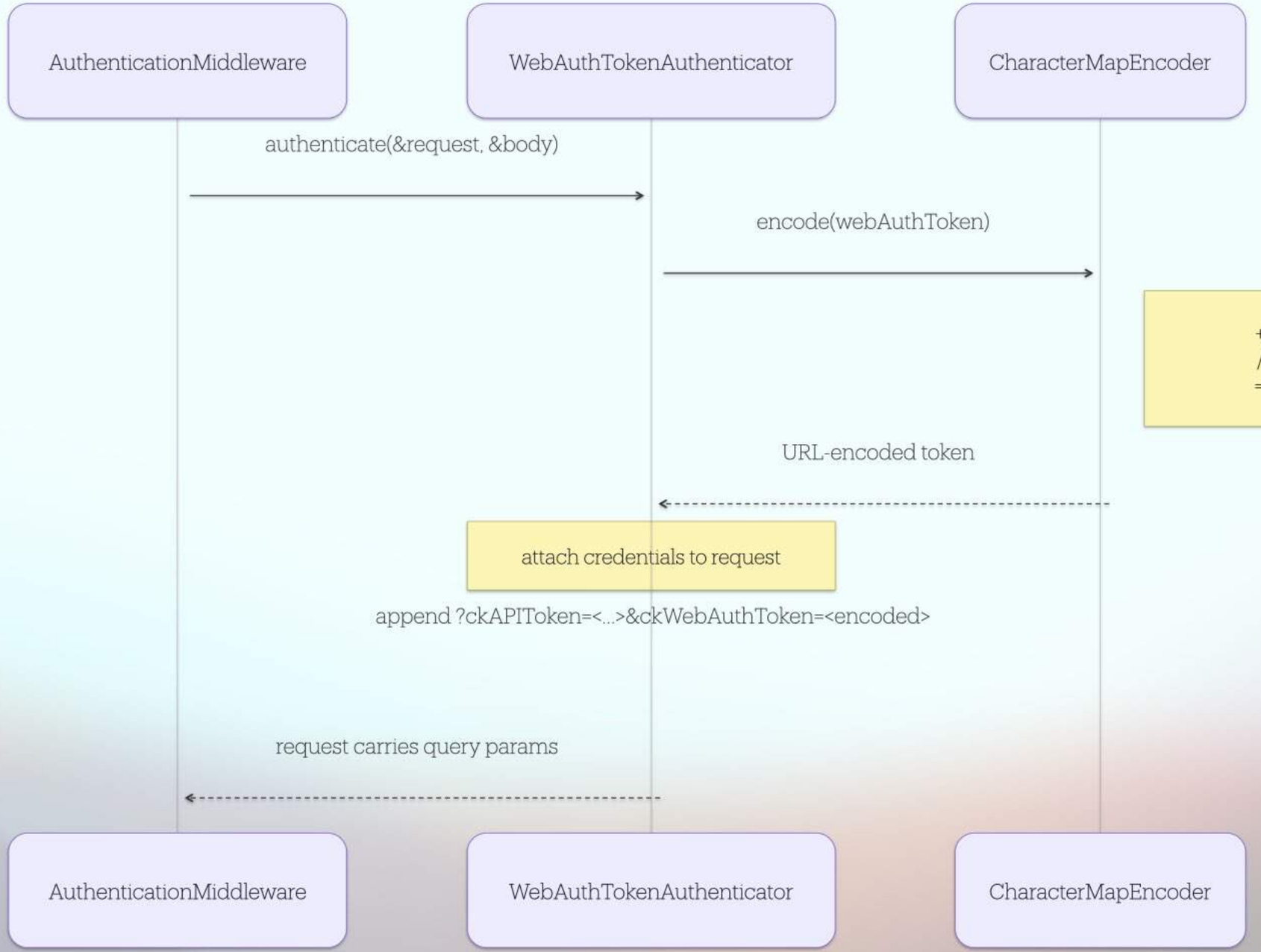
@leogdion@c.im


```swift
public func authenticate(
  request: inout HTTPRequest,
  body: inout HTTPBody?
) async throws {
  request.appendQueryItems([URLQueryItem(name: "ckAPIToken", value: token)])
}
```

@leogdion@c.im

Web Authentication

@leogdion@c.im



@leogdion@c.im

```
public func authenticate(
  request: inout HTTPRequest,
  body: inout HTTPBody?
) async throws {
  let encoded = Self.encoder.encode(webAuthToken)
  request.appendQueryItems([
    URLQueryItem(name: "ckAPIToken", value: apiToken),
    URLQueryItem(name: "ckWebAuthToken", value: encoded),
  ])
}
```

+ → %2B
/ → %2F
= → %3D

@leogdion@c.im

Server to Server

@leogdion@c.im

Monitor
Telemetry
Usage
Logs
Alerts

Data
Records
Zones
Subscriptions

Schema
Indexes – Modified
Record Types – Modified
Security Roles – Modified
History

Settings
Tokens & Keys
Sharing Fallback

Act As iCloud Account...
Deploy Schema Changes...
Import Schema...
Export Schema...
Reset Environment...
Container Permissions

Tokens & Keys

"My New Server Key" Details

Delete

Key ID: fc9f8fc677ffe615a2e28b6be189f937c093a2393e49556d7fa459497ebb7a4a

Name: My New Server Key

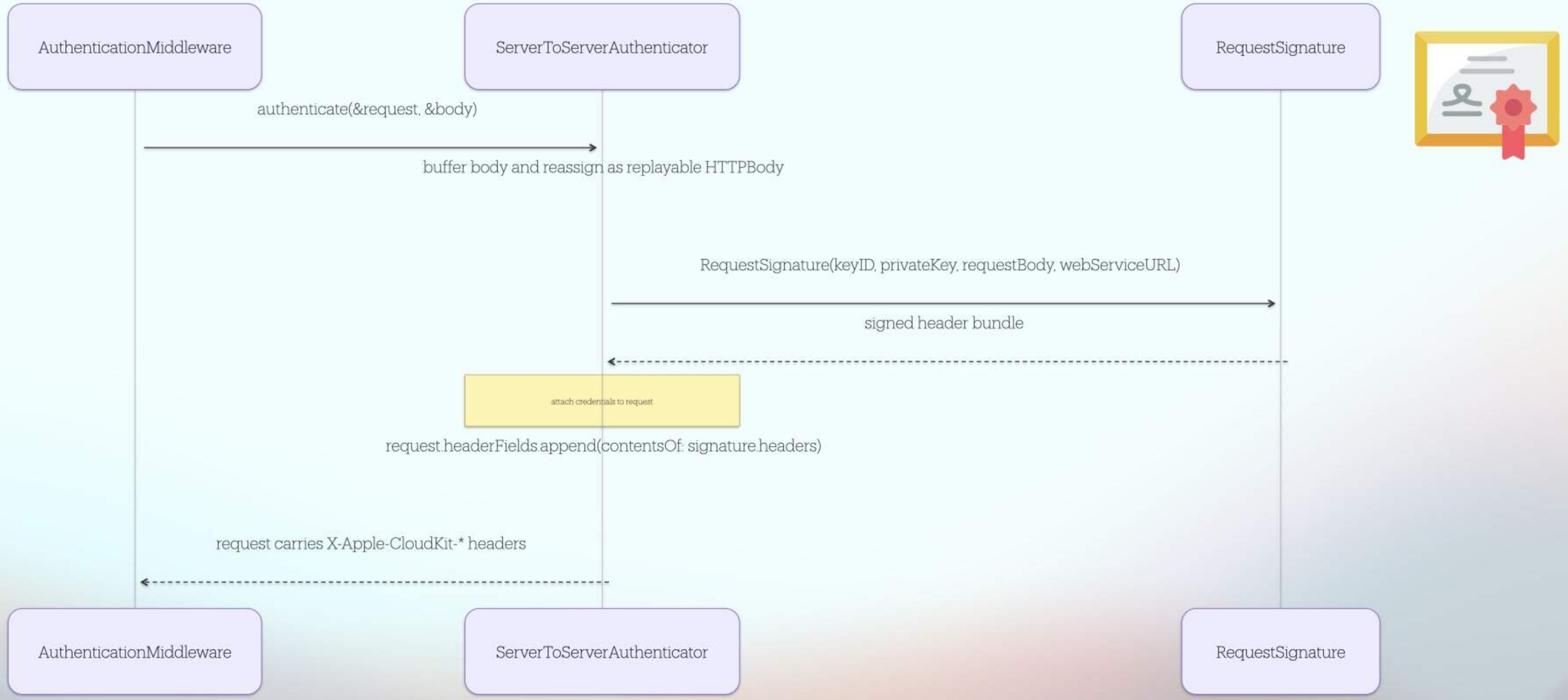
Public Key:

```
MFkwEwYHKoZIzj0CAQYIKoZIzj0DAQcDQgAETb03N/VE8czuZwq/GcSNDMw2FYUw
r/HFfi/ycjRuqK8TvEB6IpGeSKVXOMs2jYTqE9qzpQHvJN+Le13eY93ZLw==
```

Notes: This is for demo purposes

@leogdion@c.im

Name	Value
X-Apple-CloudKit-Request-KeyID	[your key ID]
X-Apple-CloudKit-Request-ISO8601Date	[the same date that was signed]
X-Apple-CloudKit-Request-SignatureV1	[base64-encoded ECDSA signature]



@leogdion@c.im

```
public func authenticate(
  request: inout HTTPRequest,
  body: inout HTTPBody?
) async throws {
  let encoded = Self.encoder.encode(webAuthToken)
  request.appendQueryItems([
    URLQueryItem(name: "ckAPIToken", value: apiToken),
    URLQueryItem(name: "ckWebAuthToken", value: encoded),
  ])
}
```

@leogdion@c.im

```
public func authenticate(
  request: inout HTTPRequest,
  body: inout HTTPBody?
) async throws {
  let bodyData = try await Data(buffering: &body, upTo: bodyBufferLimit)

  let signature = try RequestSignature(
    keyID: keyID,
    privateKey: privateKey,
    requestBody: bodyData,
    webServiceSubpath: request.path
  )

  request.headerFields.append(contentsOf: signature.headers)
}
```

@leogdion@c.im

RequestSignature

```
public init(
  keyID: String,
  privateKey: P256.Signing.PrivateKey,
  requestBody: Data?,
  webServiceSubpath: String?,
  date: Date = Date()
) throws {
  assert(
    webServiceSubpath != nil,
    "RequestSignature requires a non-nil webServiceSubpath; HTTPRequest.path was nil"
  )
  try self.init(
    keyID: keyID,
    privateKey: privateKey,
    bodyHash: SHA256.cloudKitBodyHash(of: requestBody),
    webServiceSubpath: webServiceSubpath ?? "",
    iso8601DateString: Self.iso8601String(from: date)
  )
}
```

@leogdion@c.im

RequestSignature

```
public init(
  keyID: String,
  privateKey: P256.Signing.PrivateKey,
  requestBody: Data?,
  webServiceSubpath: String?,
  date: Date = Date()
) throws {
  assert(
    webServiceSubpath != nil,
    "RequestSignaData(Self.hash(data: body)).base64EncodedString()quest.path was nil"
  )
  try self.init(
    keyID: keyID,
    privateKey: privateKey,
    bodyHash: SHA256.cloudKitBodyHash(of: requestBody),
    webServiceSubpath: webServiceSubpath ?? "",
    iso8601DateString: Self.iso8601String(from: date)
  )
}
```

SHA256.cloudKitBodyHash(of: requestBody)

@leogdion@c.im

RequestSignature

```
public init(
  keyID: String,
  privateKey: P256.Signing.PrivateKey,
  requestBody: Data?,
  webServiceSubpath: String?,
  date: Date = Date()
) throws {
  assert(
    webServiceSubpath != nil,
    "RequestSignature requires a non-nil webServiceSubpath; HTTPRequest.path was nil"
  )
  try self.init(
    keyID: keyID,
    privateKey: privateKey,
    bodyHash: SHA256.cloudKitBodyHash(of: requestBody),
    webServiceSubpath: webServiceSubpath ?? "",
    iso8601DateString: Self.iso8601String(from: date)
  )
}
```

@leogdion@c.im

RequestSignature

```
public init(
  keyID: String,
  privateKey: P256.Signing.PrivateKey,
  bodyHash: String,
  webServiceSubpath: String,
  iso8601DateString: String
) throws {
  let payload = "\(iso8601DateString):\(bodyHash):\(webServiceSubpath)"
  let signature = try privateKey.signature(for: Data(payload.utf8))

  self.init(
    keyID: keyID,
    iso8601DateString: iso8601DateString,
    signatureDerRepresentation: signature.derRepresentation
  )
}
```

@leogdion@c.im

```
public var headers: HTTPFields {
  var fields = HTTPFields()
  fields[.cloudKitRequestKeyID] = keyID
  fields[.cloudKitRequestISO8601Date] = iso8601DateString
  fields[.cloudKitRequestSignatureV1] = signatureBase64
  return fields
}
```

@leogdion@c.im

Field Type Polymorphism

@leogdion@c.im

On This Page

Types and Dictionaries

The types and dictionaries listed below are used by multiple CloudKit web service requests that correspond to types or classes in CloudKit JS and the native API.

Types

CKValue

A `CKValue` represents a field type value. The table shows the supported field types.

Schema field type	JavaScript type	Description
Asset	`Asset` dictionary	A large file that is associated with a record but stored separately. Represented by a dictionary, described in Asset Dictionary.
	`Boolean`	A `true` or `false` value.

Schema field type	JavaScript type	Description
Asset	`Asset` dictionary	A large file that is associated with a record but stored separately. Represented by a dictionary, described in Asset Dictionary.
	`Boolean`	A `true` or `false` value.
Bytes	`String`	A wrapper for byte buffers that is stored with the record. Represented by a Base64-encoded string.
Date/Time	`Number`	A date or time value. Represented as a number in milliseconds between midnight on January 1, 1970, and the specified date or time.
Double	`Number`	A double.
Int(64)	`Number`	An integer.
Location	`Location` dictionary	A geographical coordinate and altitude, represented by a dictionary, described in Location Dictionary.
Reference	`Reference` dictionary	A relationship from one object to another, represented by a dictionary, described in Reference Dictionary.
String	`String`	A text string.
List	`Array`	An array containing any of the above field types.

String

Int

NSNumber

Double

Bytes

Data

Date

Location

CLLocation

Reference

CKReference

Asset

CKAsset

List

Array<>

@leogdion@c.im

String

Int

Double

Bytes

Data

Date

Location

Location

Reference

Reference

Asset

Asset

List

Array<>

@leogdion@c.im

```swift
public enum FieldValue: Codable, Equatable, Sendable {
  case string(String)
  case int64(Int)
  case double(Double)
  case bytes(Data)
  case date(Date)  // Date/time value
  case location(Location)
  case reference(Reference)
  case asset(Asset)
  case list([FieldValue])
}
```

@leogdion@c.im

```swift
/// Reference dictionary as defined in CloudKit Web Services
public struct Reference: Codable, Equatable, Sendable {
  /// Reference action types supported by CloudKit
  public enum Action: String, Codable, Sendable {
    case deleteSelf = "DELETE_SELF"
    case none = "NONE"
  }

  /// The record name being referenced
  public let recordName: String
  /// The action to take (DELETE_SELF, NONE, or nil)
  public let action: Action?

  /// Initialize a reference value
  public init(recordName: String, action: Action? = nil) {
    self.recordName = recordName
    self.action = action
  }
}
```

@leogdion@c.im

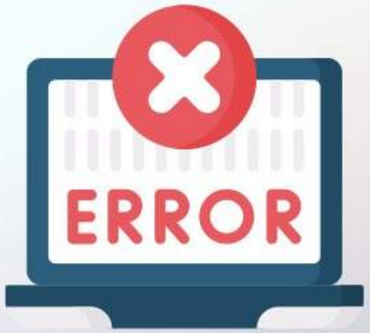
```swift
/// Asset dictionary as defined in CloudKit Web Services
public struct Asset: Codable, Equatable, Sendable {
  /// The file checksum
  public let fileChecksum: String?
  /// The file size in bytes
  public let size: Int64?
  /// The reference checksum
  public let referenceChecksum: String?
  /// The wrapping key for encryption
  public let wrappingKey: String?
  /// The upload receipt
  public let receipt: String?
  /// The download URL
  public let downloadURL: String?

  /// Initialize an asset value
  public init(
    fileChecksum: String? = nil,
    size: Int64? = nil,
    referenceChecksum: String? = nil,
    wrappingKey: String? = nil,
    receipt: String? = nil,
    downloadURL: String? = nil
  ) {
    self.fileChecksum = fileChecksum
    self.size = size
    self.referenceChecksum = referenceChecksum
    self.wrappingKey = wrappingKey
    self.receipt = receipt
    self.downloadURL = downloadURL
  }
}
```

@leogdion@c.im

```yaml
FieldValueRequest:
  properties:
    value:
      oneOf: [StringValue, Int64Value, DoubleValue, BytesValue,
              DateValue, LocationValue, ReferenceValue, AssetValue, ListValue]
    type:
      enum: [STRING_LIST, INT64_LIST, DOUBLE_LIST, ...]  # List element types only
```

@leogdion@c.im

Error Handling



@leogdion@c.im

Error Codes

This table contains the server errors that may occur when posting requests. The error codes appear in the response.

Error	Description	HTTP status code
`ACCESS_DENIED`	You don't have permission to access the endpoint, record, zone, or database.	403
`ATOMIC_ERROR`	An atomic batch operation failed.	400
`AUTHENTICATION_FAILED`	Authentication was rejected.	401
`AUTHENTICATION_REQUIRED`	The request requires authentication but none was provided.	421
`BAD_REQUEST`	The request was not valid.	400
`CONFLICT`	The `recordChangeTag` value expired. (Retry the request with the latest tag.)	409
`EXISTS`	The resource that you attempted to create already exists.	409
`INTERNAL_ERROR`	An internal error occurred.	500

Error	Description	HTTP status code
`ACCESS_DENIED`	You don't have permission to access the endpoint, record, zone, or database.	403
`ATOMIC_ERROR`	An atomic batch operation failed.	400
`AUTHENTICATION_FAILED`	Authentication was rejected.	401
`AUTHENTICATION_REQUIRED`	The request requires authentication but none was provided.	421
`BAD_REQUEST`	The request was not valid.	400
`CONFLICT`	The `recordChangeTag` value expired. (Retry the request with the latest tag.)	409
`EXISTS`	The resource that you attempted to create already exists.	409
`INTERNAL_ERROR`	An internal error occurred.	500
`NOT_FOUND`	The resource was not found.	404
`QUOTA_EXCEEDED`	If accessing the public database, you exceeded the app's quota. If accessing the private database, you exceeded the user's iCloud quota.	413
`THROTTLED`	The request was throttled. Try the request again later.	429
`TRY_AGAIN_LATER`	An internal error occurred. Try the request again.	503
`VALIDATING_REFERENCE_ERROR`	The request violates a validating reference constraint.	412
`ZONE_NOT_FOUND`	The zone specified in the request was not found.	404

```
{
  "uuid": "a1b2c3d4-...",
  "serverErrorCode": "AUTHENTICATION_FAILED",
  "reason": "The request requires authentication."
}
```

```yaml
ErrorResponse:
  properties:
    uuid: { type: string }
    serverErrorCode:
      type: string
      enum: [ACCESS_DENIED, ATOMIC_ERROR, AUTHENTICATION_FAILED, ...]
    reason: { type: string }
    redirectURL: { type: string }
```

@leogdion@c.im

```
// The error code enum
internal enum serverErrorCodePayload: String,
Codable, CaseIterable {
    case ACCESS_DENIED, ATOMIC_ERROR,
AUTHENTICATION_FAILED, ...
}

// The error response struct
internal struct ErrorResponse: Codable {
    internal var uuid: String?
    internal var serverErrorCode:
serverErrorCodePayload?
    internal var reason: String?
    internal var redirectURL: String?
}
```

@leogdion@c.im

LD
Leo Dion
May 11, 2026 at 11:27

discoverAllUserIdentities returns HTTP 500 INTERNAL_ERROR via REST and CloudKitJS

FB22754466

Recent Similar Reports: None
Resolution: Investigation complete - Unable to diagnose with current information

Basic Information

Please provide a descriptive title for your feedback:
discoverAllUserIdentities returns HTTP 500 INTERNAL_ERROR via REST and CloudKitJS

Which platform is most relevant for your report?
Web & Services

Which technology does your report involve?
CloudKit

What type of feedback are you reporting?
Incorrect/Unexpected Behavior

Where does the issue occur?
Not answered

Description

Please describe the issue and what steps we can take to reproduce it:
discoverAllUserIdentities returns HTTP 500 INTERNAL_ERROR via REST and CloudKitJSApple's discoverAllUserIdentities endpoint is broken on the server. It is reproducibly returning HTTP 500 with serverErrorCode: "INTERNAL_ERROR" from both the documented REST endpoint and from Apple's own cloudkit.js library. The failure happens after authentication succeeds (auth tokens are correctly refreshed in the response headers), so it is a server-side handler crash rather than an auth or routing problem.

Steps to reproduce:

REST path:

GET https://api.apple-cloudkit.com/database/1/{container}/development/public/users/discover
  ?ckAPIToken={api-token}
  &ckWebAuthToken={character-map-encoded-web-auth-token}
CloudKitJS path: in a browser authenticated to a CloudKit container, call

REST path:

GET https://api.apple-cloudkit.com/database/1/{container}/development/public/users/discover
  ?ckAPIToken={api-token}
  &ckWebAuthToken={character-map-encoded-web-auth-token}
CloudKitJS path: in a browser authenticated to a CloudKit container, call

container.discoverAllUserIdentities()
Expected: 200 OK with a users array (per the archived REST docs and the CloudKitJS docs).

Actual: 500 Internal Server Error with body:

```
{
  "uuid": "<correlation-uuid>",
  "serverErrorCode": "INTERNAL_ERROR"
}
```

CloudKitJS surfaces the same failure as a CKError { serverErrorCode: "INTERNAL_ERROR" } rejection.

Diagnostic evidence (rules out client-side wire-shape issues):

OPTIONS on the same URL returns 200 OK with Access-Control-Allow-Methods: GET, POST, PUT, DELETE — the GET method is registered on this URL.
A typo'd path on the same prefix (e.g. /users/discoverAll) returns 404 NOT_FOUND with "could not find handler for endpoint 'GET <name>'" — so unknown paths return NOT_FOUND, not INTERNAL_ERROR. Our path is recognized.
Sending GET /users/discover against private/ or shared/ databases returns 400 BAD_REQUEST with "endpoint not applicable in the database type 'privatedb'/'sharedb'" — confirming the public-database segment is correct.
POST /users/discover against the identical URL with the identical auth credentials reaches the handler and runs body validation (returns 400 BAD_REQUEST: discover request contains an invalid user id for an empty lookupInfos array).
Auth succeeds even on the failing GET requests: every 500 response includes a refreshed X-Apple-CloudKit-Web-Auth-Token header, which CloudKit only emits after authentication is accepted.
Apple's own CloudKitJS (served from cdn.apple-cloudkit.com/ck/2/cloudkit.js) sends the byte-for-byte equivalent request — the request builder defaults to GET when no payload is set, and discoverAllUserIdentities never calls setPayload. Calling container.discoverAllUserIdentities() from an authenticated browser produces the same 500.
Affected scope: Both the REST API and CloudKitJS. Sibling endpoints on the same path prefix (users/caller, POST users/discover) work normally with the same auth and host.

Apple correlation UUIDs from the affected container iCloud.com.brightdigit.MistDemo, environment development:

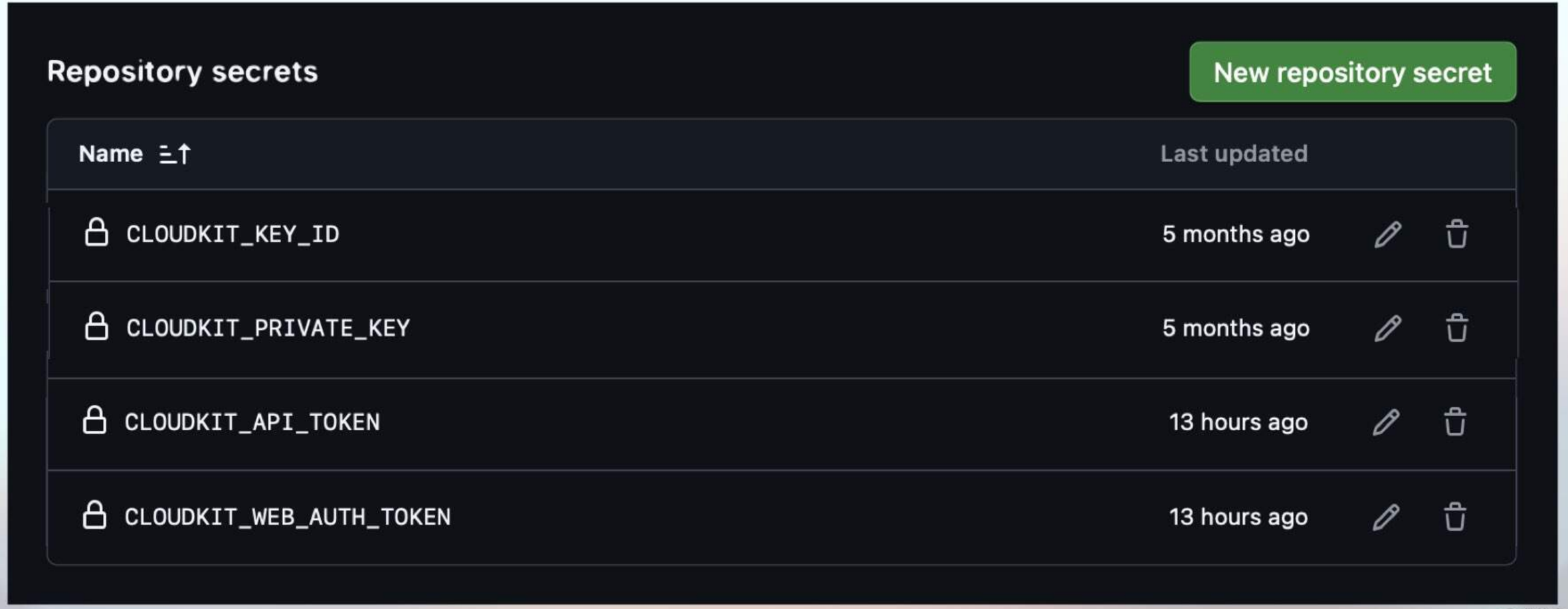
Source	Method	UUID	Time (UTC)
REST	GET	fd63e695-214f-4ad7-90d0-f78966809285	2026-05-08T19:23:59Z
REST	GET	6c09078c-71d8-46a1-907d-bcb5604b6306	2026-05-08T19:23:59Z
REST	GET	deb4efc6-fc08-41b3-a2e1-fe6fa6585395	2026-05-08T19:24:00Z
CloudKitJS	GET	26a433f0-d9df-483a-b086-9b35b527ce8a	2026-05-08T19:43:20Z

Frequency: 100% reproducible.

Suggested resolution: If the endpoint has been retired, return a documented status (410 GONE or 400 with an explicit serverErrorCode such as ENDPOINT_RETIRED) and update the archived REST docs / CloudKitJS docs accordingly. Returning INTERNAL_ERROR to a request with the documented shape is misleading and prevents downstream code from handling the deprecation cleanly.

Deployment

@leogdion@c.im



@leogdion@c.im

Repository secrets

New repository secret

Name	Last updated
CLOUDKIT_KEY_ID	5 months ago
CLOUDKIT_PRIVATE_KEY_BASE64	13 hours ago
CLOUDKIT_API_TOKEN	13 hours ago
CLOUDKIT_WEB_AUTH_TOKEN	13 hours ago

@leogdion@c.im

```yaml
name: Scheduled CloudKit Sync (Development)

on:
  schedule:
    - cron: '17 2 * * *'   # 02:17 UTC
    - cron: '43 10 * * *'  # 10:43 UTC
    - cron: '29 18 * * *'  # 18:29 UTC

  # Manual trigger for testing
  workflow_dispatch:

jobs:
  sync-dev:
    name: Sync to CloudKit (Development)
    runs-on: ubuntu-latest
    timeout-minutes: 30

    permissions:
      contents: read  # Read repository code

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: CloudKit Sync
        uses: ./.github/actions/cloudkit-sync
        with:
          environment: development
          container-id: iCloud.com.brightdigit.Bushel
          cloudkit-key-id: ${{ secrets.CLOUDKIT_KEY_ID }}
          cloudkit-private-key: ${{ secrets.CLOUDKIT_PRIVATE_KEY }}
```

@leogdion@c.im

```yaml
name: 'CloudKit Sync Action'

inputs:
  environment:
    description: 'CloudKit environment (development or production)'
    required: true
  container-id:
    description: 'CloudKit container ID'
    required: true
  cloudkit-key-id:
    description: 'CloudKit S2S key ID'
    required: true
  cloudkit-private-key:
    description: 'CloudKit S2S private key (PEM content)'
    required: true

runs:
  using: "composite"
  steps:
    - name: Download pre-built binary (if available)
      id: download-binary
      uses: dawidd6/action-download-artifact@v3
      continue-on-error: true  # Don't fail if artifact is missing
      with:
        workflow: bushel-cloud-build.yml
        workflow_conclusion: success
        name: bushel-cloud-binary
        path: ./binary
        branch: ${{ github.ref_name }}

    - name: Build binary (fallback if artifact unavailable)
      if: steps.download-binary.outcome != 'success'
      shell: bash
      run: |
        # Build using Swift 6.2 (matches build workflow)
        docker run --rm -v "$PWD:/workspace" -w /workspace swift:6.2-noble \
          swift build -c release --static-swift-stdlib
```

```
      workflow: bushel-cloud-build.yml
      workflow_conclusion: success
      name: bushel-cloud-binary
      path: ./binary
      branch: ${{ github.ref_name }}

  - name: Build binary (fallback if artifact unavailable)
    if: steps.download-binary.outcome != 'success'
    shell: bash
    run: |
      # Build using Swift 6.2 (matches build workflow)
      docker run --rm -v "$PWD:/workspace" -w /workspace swift:6.2-noble \
        swift build -c release --static-swift-stdlib

      # Copy binary to expected location
      mkdir -p ./binary
      cp .build/release/bushel-cloud ./binary/

  - name: Make binary executable
    shell: bash
    run: chmod +x ./binary/bushel-cloud

  - name: Run CloudKit sync with change tracking
    shell: bash
    env:
      CLOUDKIT_KEY_ID: ${{ inputs.cloudkit-key-id }}
      CLOUDKIT_PRIVATE_KEY: ${{ inputs.cloudkit-private-key }}
      CLOUDKIT_ENVIRONMENT: ${{ inputs.environment }}
      CLOUDKIT_CONTAINER_ID: ${{ inputs.container-id }}
    run: |
      ./binary/bushel-cloud sync \
        --verbose \
        --container-identifier "$CLOUDKIT_CONTAINER_ID"
```

@leogdion@c.im

CloudKit Restore Images (Debug)

iCloud.com.brightdigit.Bushel

public database · RestoreImage · loading…

Reload

Querying the public database…

What's Next?

@leogdion@c.im

Open 13 Closed 230 Author Labels Projects Milestones Assignees Types Newest

Stream large CloudKit assets instead of buffering whole-file Data enhancement library-api performance
#474 · leogdion opened 4d ago

Verify Windows Support
#457 · leogdion opened 1w ago

Verify Android Support
#456 · leogdion opened 1w ago

Research Encrypted Fields library-api
#392 · leogdion opened on May 26 · v1.1.0 4

Enhancement: Implement automatic retry logic for expired tokens enhancement
#203 · leogdion opened on Jan 9 · v1.1.0

Enhancement: Add progress tracking for large asset uploads enhancement

Enhancement: Add progress tracking for large asset uploads enhancement
#202 · leogdion opened on Jan 9 · v1.1.0

Enhancement: Support batch asset upload (multiple files) enhancement
#201 · leogdion opened on Jan 9 · v1.1.0

Enhancement: Add AsyncSequence wrapper for fetchRecordChanges streaming enhancement
#200 · leogdion opened on Jan 9 · v1.1.0

Add Foundation Predicate Support for Type-Safe CloudKit Queries (iOS 17+) enhancement
#151 · leogdion opened on Nov 13, 2025 · v1.1.0

#392 · leogdion opened on May 26 · v1.1.0 4

- [ ] **Enhancement: Implement automatic retry logic for expired tokens** enhancement
  #203 · leogdion opened on Jan 9 · v1.1.0
- [ ] **Enhancement: Add progress tracking for large asset uploads** enhancement
- [ ] **Enhancement: Add progress tracking for large asset uploads** enhancement
  #202 · leogdion opened on Jan 9 · v1.1.0
- [ ] **Enhancement: Support batch asset upload (multiple files)** enhancement
  #201 · leogdion opened on Jan 9 · v1.1.0
- [ ] **Enhancement: Add AsyncSequence wrapper for fetchRecordChanges streaming** enhancement
  #200 · leogdion opened on Jan 9 · v1.1.0
- [ ] **Add Foundation Predicate Support for Type-Safe CloudKit Queries (iOS 17+)** enhancement
  #151 · leogdion opened on Nov 13, 2025 · v1.1.0
- [ ] **Add KeyPath-based QuerySort API for Type-Safe Sorting** enhancement
  #150 · leogdion opened on Nov 13, 2025 · v1.1.0
- [ ] **Add KeyPath-based QueryFilter API for Type-Safe Filtering** enhancement
  #149 · leogdion opened on Nov 13, 2025 · v1.1.0
- [ ] **Feature: Implement AsyncSequence for streaming query results**
  #147 · leogdion opened on Nov 13, 2025 · v1.1.0

- [ ] **Add CloudKit Schema Management APIs (cktool/cktooljs functionality)**
  #135 · leogdion opened on Nov 4, 2025 · v1.1.0







@leogdion@c.im

One More Thing...

Bushel

Virtualization for App Developers

getbushel.app

Download on the
Mac App Store

AtLeast

Passive Timer for Apple Watch

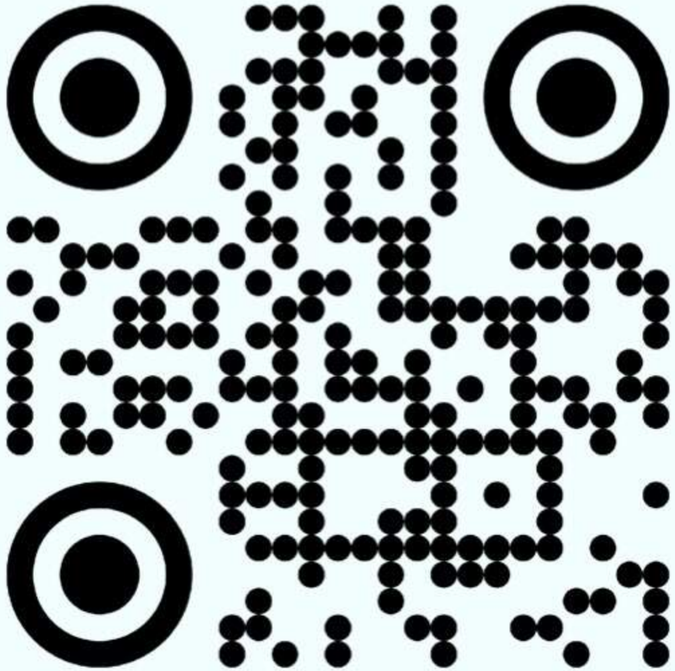
atleast.app

Join the beta on
TestFlight

@leogdion@c.im

leo@brightdigit.com

Empower Apps



linktr.ee/leogdion

github.com/brightdigit/MistKit

@leogdion@c.im